2

Unit 2: Combinational and Sequential Logic

Computer Organization and Architecture

CHANDRAS EDU

Never stop learning

Agenda

2.1 Combinational Circuits
2.2 Analysis Procedure
2.3 Design Procedure
2.4 Binary Adder
2.5 Binary Subtractor
2.6 Decimal (BCD) Adder
2.7 Binary Multiplier
2.8 Magnitude Comparator
2.9 Decoders
2.10 Encoders
2.11 Multiplexers
2.12 Sequential Circuits
2.13 Latches
2.14 Flip-Flops
2.15 Analysis of Clocked Sequential Circuits
2.16 Registers
2.17 Shift Registers
2.18 Ripple Counters
2.19 Synchronous Counters
2.20 Other Counters

2.1 Combinational Circuits

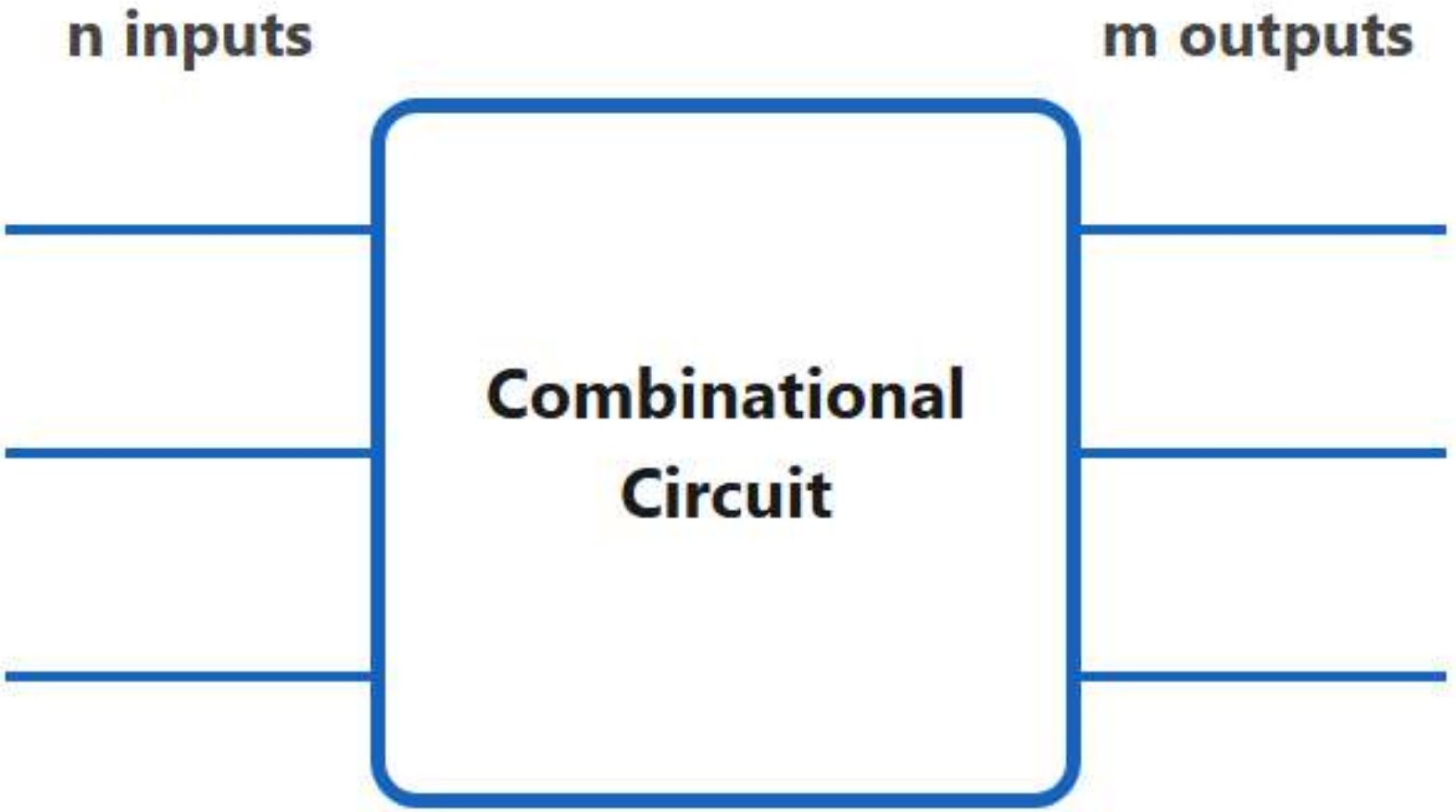
IN SIMPLE TERMS

A combinational circuit has no memory: its outputs depend only on the inputs right now.

Combinational circuits

- A **combinational circuit** is a group of logic gates whose output depends **only on the current inputs**: never on past inputs and never on a clock
- It has **no feedback**; no gate output loops back to an earlier gate
- If flip-flops store the circuit's past output and feed it back, it is a **sequential circuit**
- The output settles shortly after the inputs change: **propagation delay**
- For n inputs there are $\mathbf{2^n}$ input combinations, so a truth table fully describes the circuit; 3 inputs give $2^3 = 8$ rows
- Every row needs a defined output

n inputs, m outputs



The general shape every combinational circuit takes — n inputs in, m outputs out, decided purely by the present input combination.

2.2 Analysis Procedure

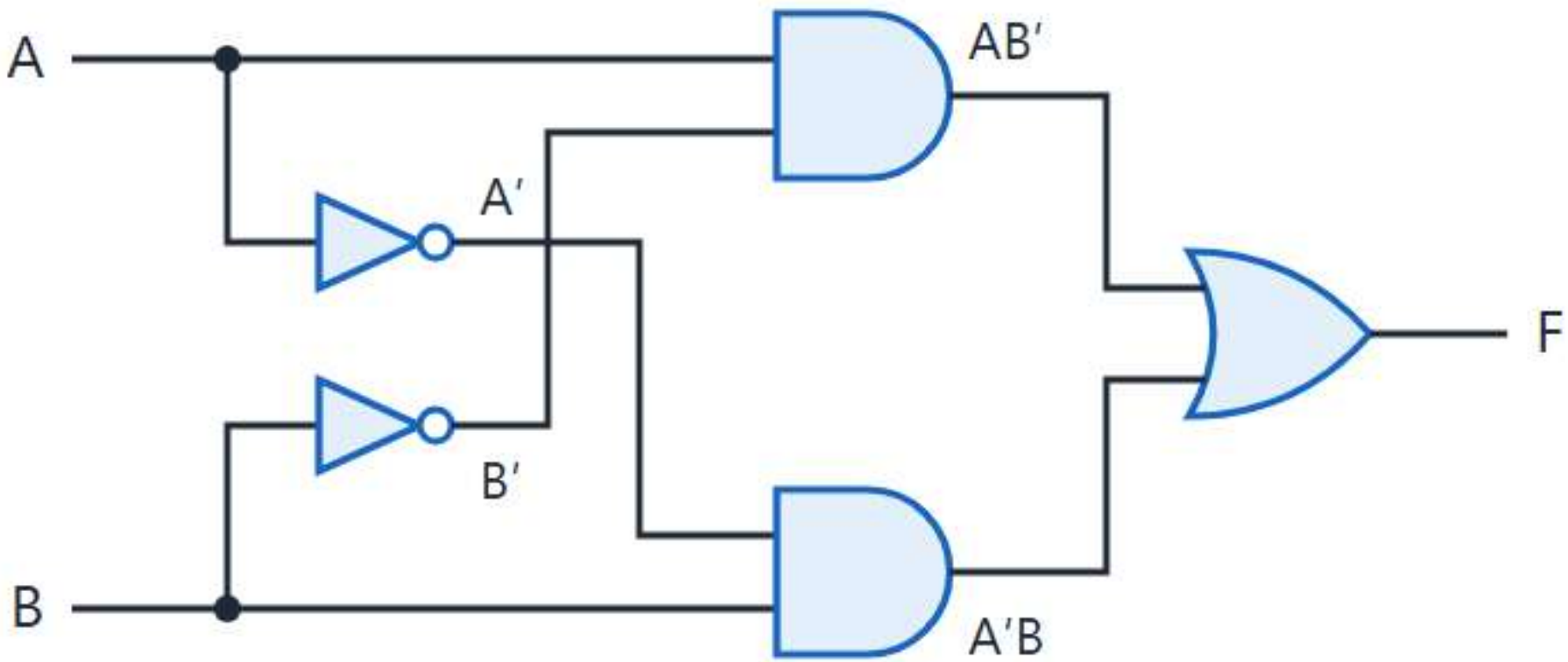
IN SIMPLE TERMS

Analysis goes from circuit to function: name every gate output, write its expression, then substitute until only the inputs remain.

The analysis procedure

- **1. Label** every gate output with a variable name, from the inputs toward the output
- **2. Write** the Boolean expression for each labelled signal
- **3. Substitute** until the final output uses only the original inputs
- **4. Tabulate:** build the truth table by evaluating the final expression for all 2^n input combinations, if needed
- Most common mistake: skipping step 1 and writing the final expression straight from the diagram

Worked example 1: a 2-input circuit



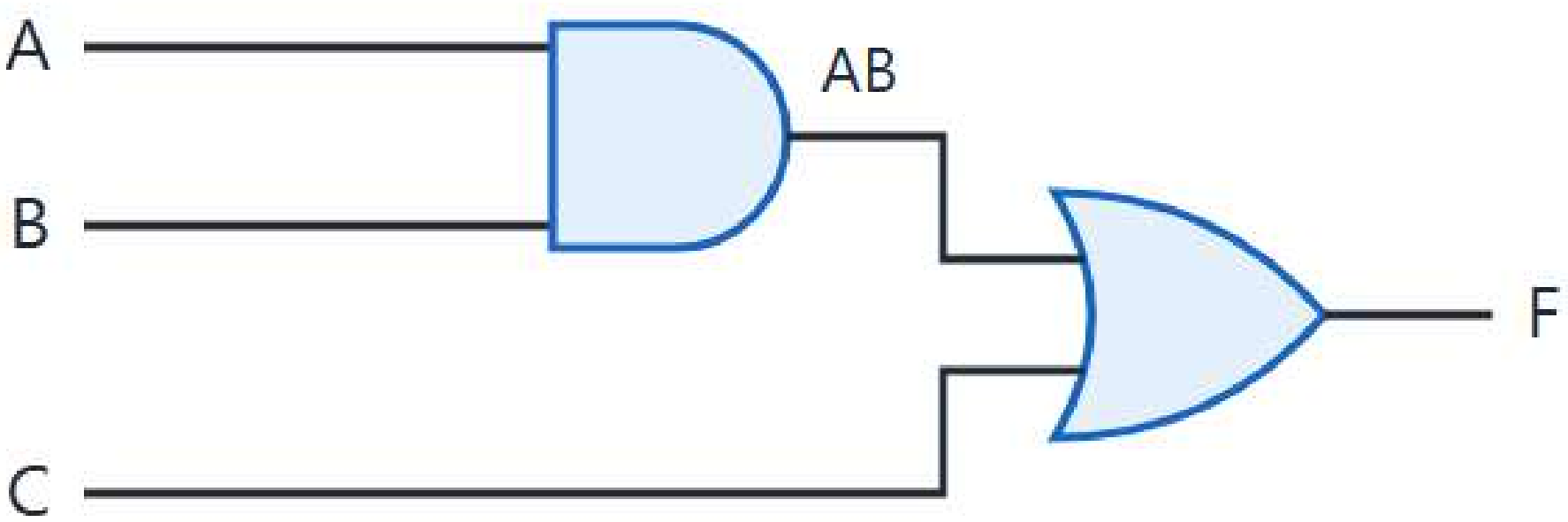
Circuit for Worked Example 1: F = A’B + AB’.

Example 1: checking all four input combinations

F = A’B + AB’. **F is 1 exactly when A and B differ:** the XOR function, built here from AND, OR and NOT gates.

A	B	A’B	AB’	F = A’B + AB’
0	0	0	0	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	0

Worked example 2: a 3-input circuit



Circuit for Worked Example 2: F = AB + C.

Example 2: F = AB + C


```
Step 1: label the AND gate output   w1 = AB
Step 2: the OR gate combines w1 and C   F = w1 + C
Step 3: substitute   F = AB + C
```

- F is 1 whenever C is 1, or whenever both A and B are 1
- Labelling w1 first made this easier than reading the whole expression at once

2.3 Design Procedure

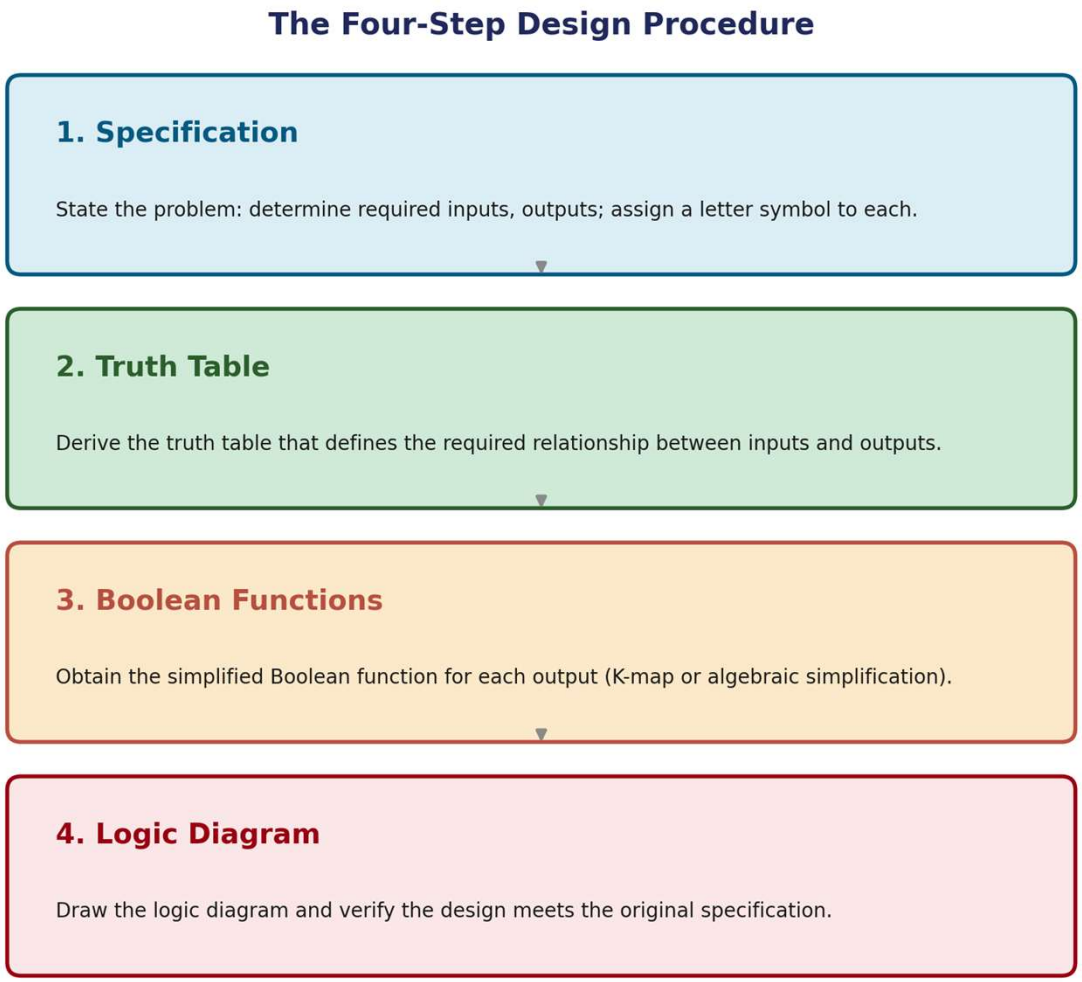
IN SIMPLE TERMS

Design goes from words to circuit: truth table, simplify, draw, then check the drawing gives the same truth table.

The design procedure

- **1. Specification:** state the problem; decide the number of inputs and outputs and give each a letter symbol
- **2. Truth table:** the required output for every input combination
- **3. Simplify:** the simplified Boolean function for each output, by K-map or algebra
- **4. Draw and verify:** draw the logic diagram and re-derive its truth table to compare with step 2

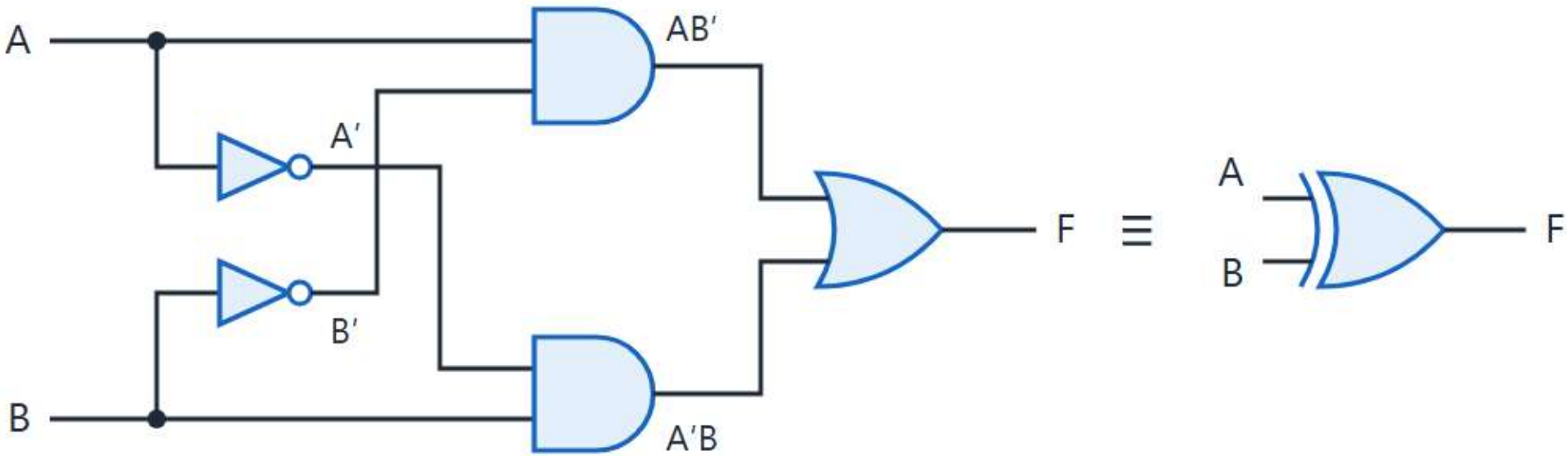
The four-step design procedure



- **1. Specification:** inputs, outputs and a letter symbol for each
- **2. Truth table:** the required output for every input combination
- **3. Simplify:** K-map or algebra
- **4. Draw and verify:** re-derive the truth table and compare

The four-step design procedure, applied to every combinational circuit in this unit.

Example 1: the same circuit, and the XOR gate



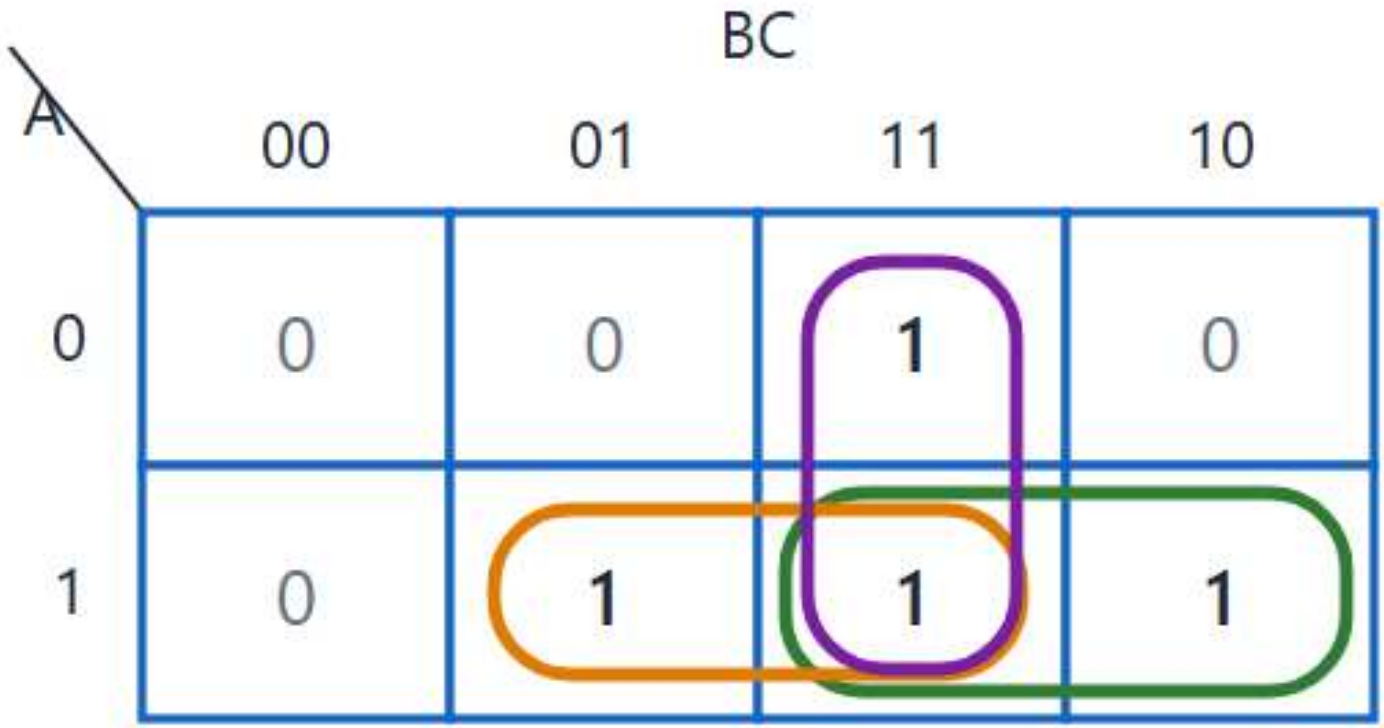
Same circuit as in Analysis Procedure — and the single XOR gate it's equivalent to

Example 2: 3-input majority function

F = 1 when 2 or more inputs are 1: F(A,B,C) = Σm(3,5,6,7).

A	B	C	F	minterm
0	0	0	0	m0
0	0	1	0	m1
0	1	0	0	m2
0	1	1	1	m3
1	0	0	0	m4
1	0	1	1	m5
1	1	0	1	m6
1	1	1	1	m7

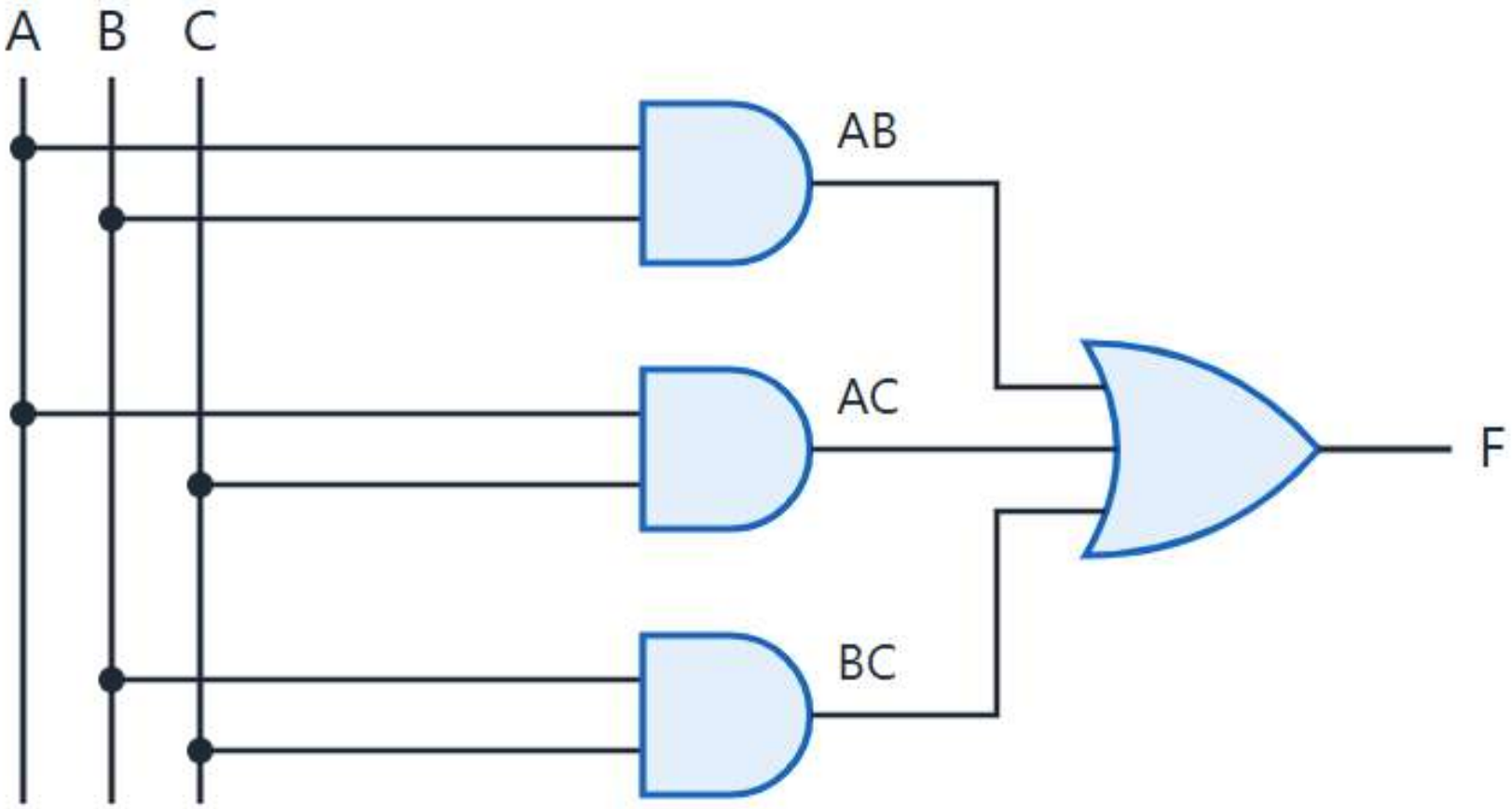
Majority function: K-map



m6 + m7 → AB m5 + m7 → AC m3 + m7 → BC

K-map for the majority function: three overlapping pairs give F = AB + AC + BC.

Majority circuit



Majority circuit: three AND gates feeding one three-input OR gate.

2.4 Binary Adder

IN SIMPLE TERMS

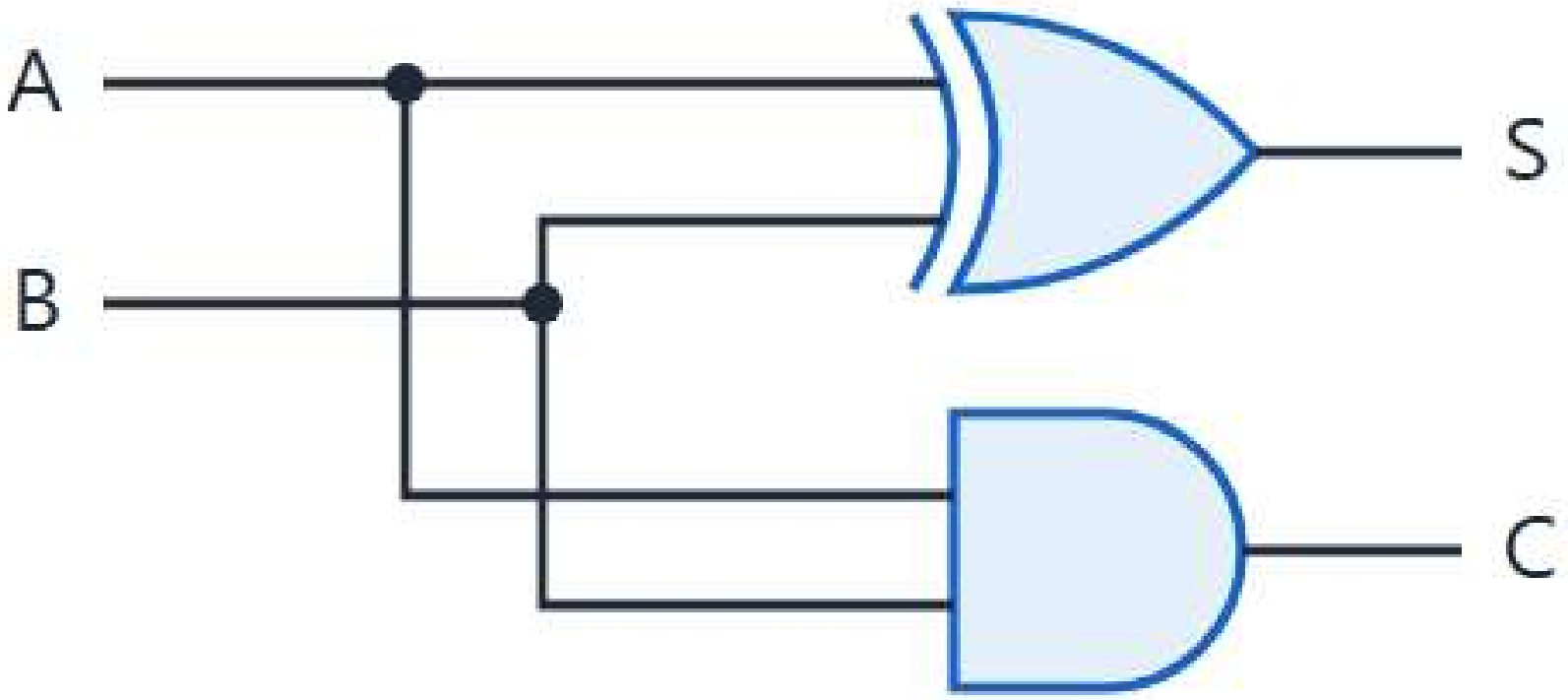
A half adder adds two bits; a full adder also adds the carry from the bit before; chaining full adders adds whole numbers.

Half adder

$S = A \oplus B$ $C = A \cdot B$. A half adder has no carry-in, so it works only for the lowest bit.

A	B	S (Sum)	C (Carry)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half adder



Half adder — one XOR gate makes the sum, one AND gate makes the carry.

Full adder

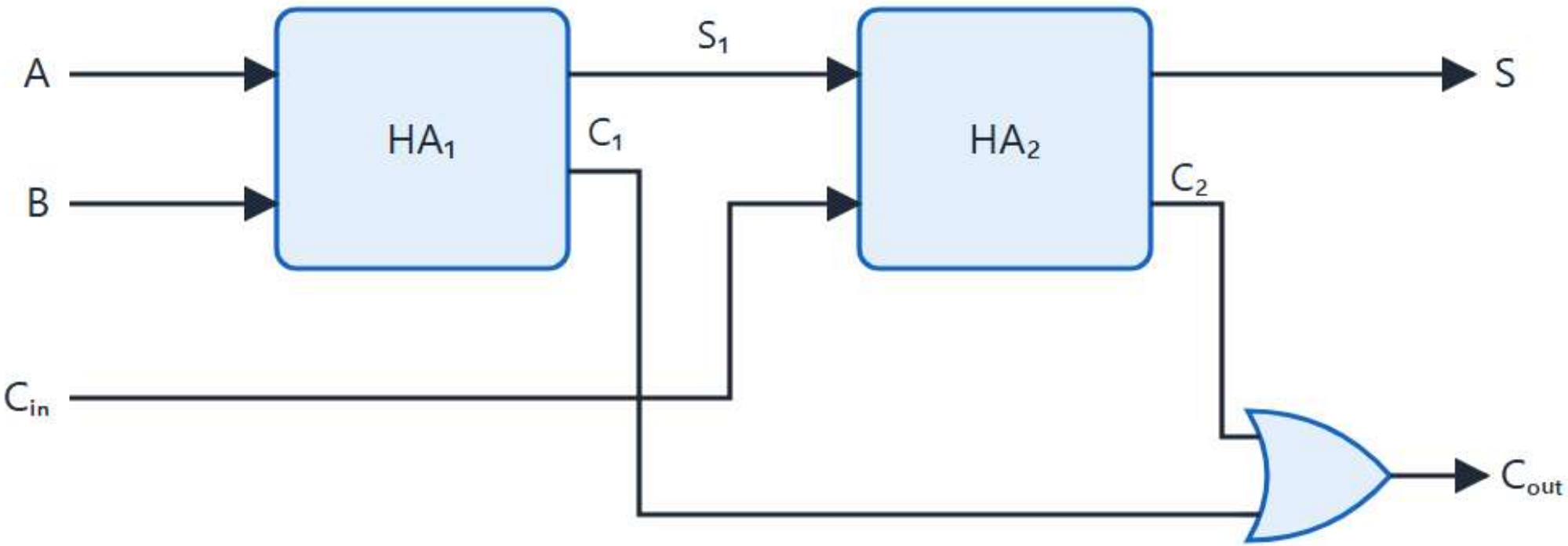
$S = A \oplus B \oplus C_{in}$ $C_{out} = AB + BC_{in} + AC_{in}$

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Building the full adder

- Reuse two half adders plus one OR gate
- **HA_1** adds A and B: partial sum S_1 and carry C_1
- **HA_2** adds S_1 and C_{in}: final sum S and a second carry C_2
- C_1 and C_2 can never both be 1, so OR-ing them gives the correct C_{out}

Full adder from two half adders

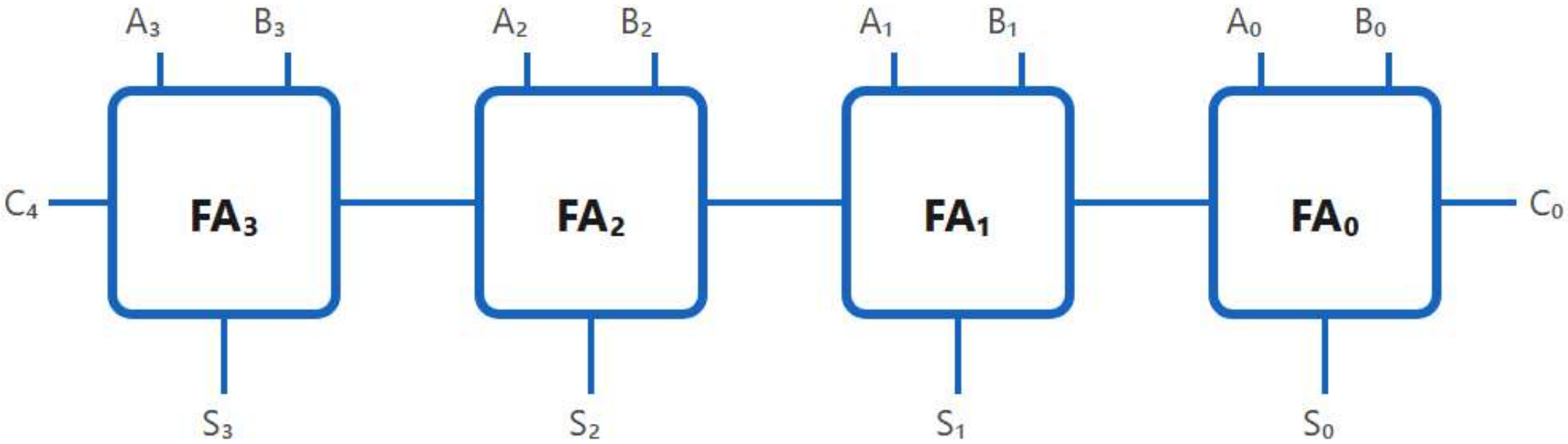


A full adder built from two half adders and one OR gate. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.8.

The ripple carry adder

- A **binary adder** adds two multi-bit numbers: connect full adders in a chain
- Each stage’s carry-out feeds the next stage’s carry-in
- A carry made at the lowest bit can ripple through every stage: a **ripple carry adder**
- Adding two n-bit numbers needs n full adders
- The lowest stage can use a full adder with its carry-in tied to 0, which keeps every stage identical

4-bit ripple carry adder



A 4-bit ripple carry adder — four full adders, each carry-out feeding the next carry-in. C_0 is the carry into the very first (rightmost) stage, normally tied to 0; C_4 is the final carry-out.

Worked example: A = 1001 (9) + B = 0110 (6)

Work right to left. The result is $01111_2 = 15$; bit 4 is the final carry-out $C_4 = 0$. Check: 9 + 6 = 15 ✓

Bit position	4	3	2	1	0
Carry into this bit	—	0	0	0	0
A	—	1	0	0	1
B	—	0	1	1	0
Result	**0**	**1**	**1**	**1**	**1**

2.5 Binary Subtractor

IN SIMPLE TERMS

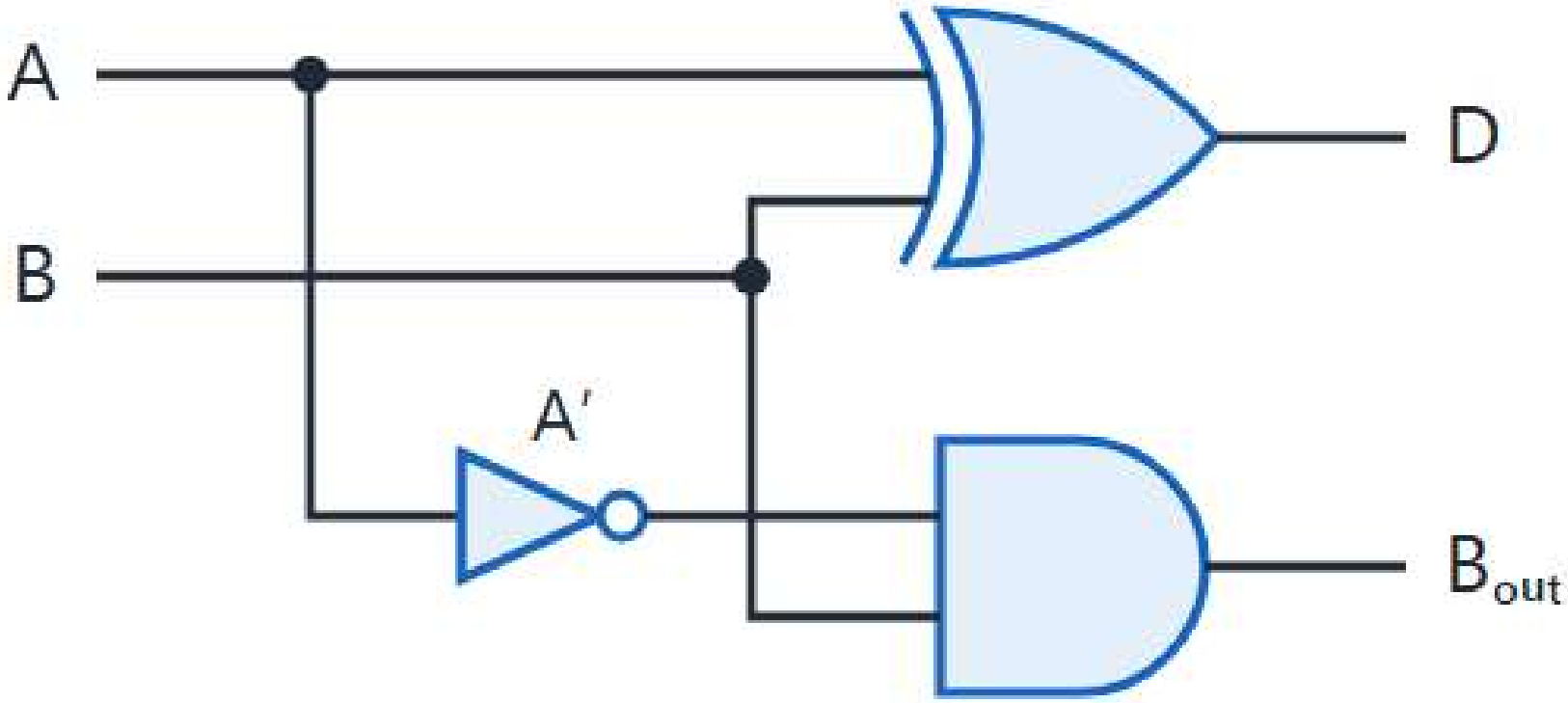
Subtraction is addition in disguise: invert B and add 1, so one adder with an XOR on each B input can do both.

Half subtractor

D = A $\oplus$ B B_{out} = A'B. A borrow of 1 means this position could not subtract on its own and had to borrow from the next position.

A	B	D (Difference)	B_{out} (Borrow)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Half subtractor



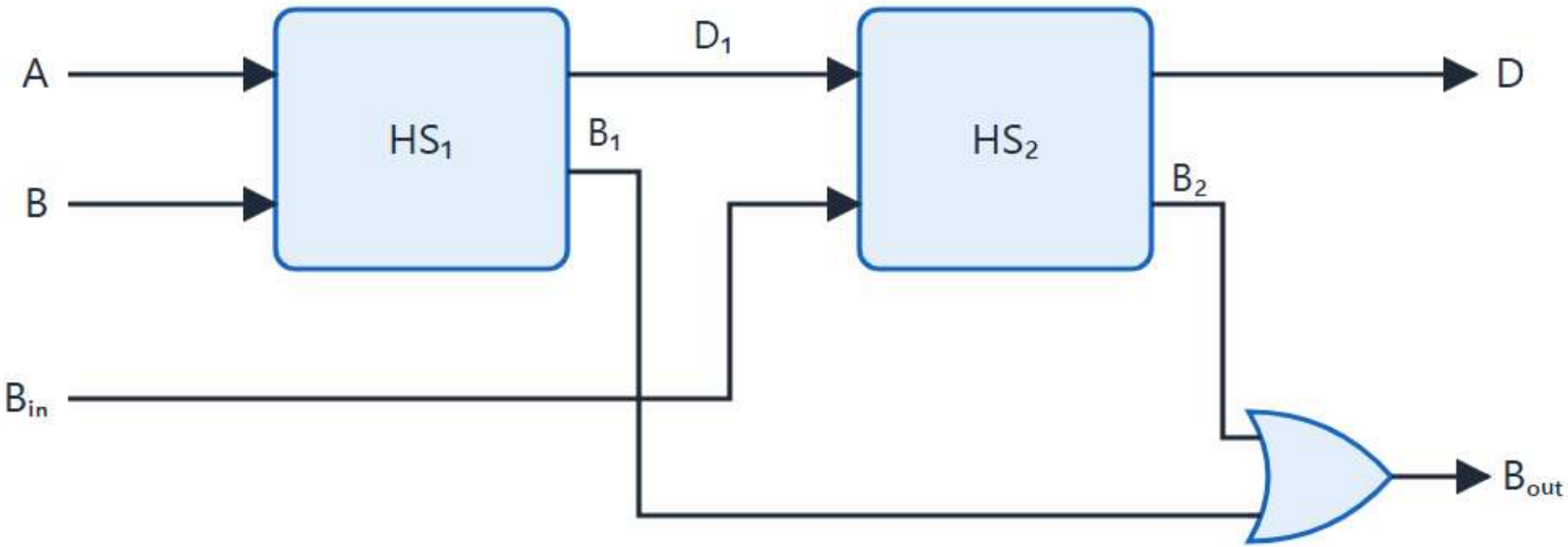
Half subtractor — one XOR gate makes the difference, one NOT + AND gate makes the borrow.

Full subtractor

$D = A \oplus B \oplus B_{in}$ $B_{out} = A'B + A'B_{in} + BB_{in}$

A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Full subtractor from two half subtractors

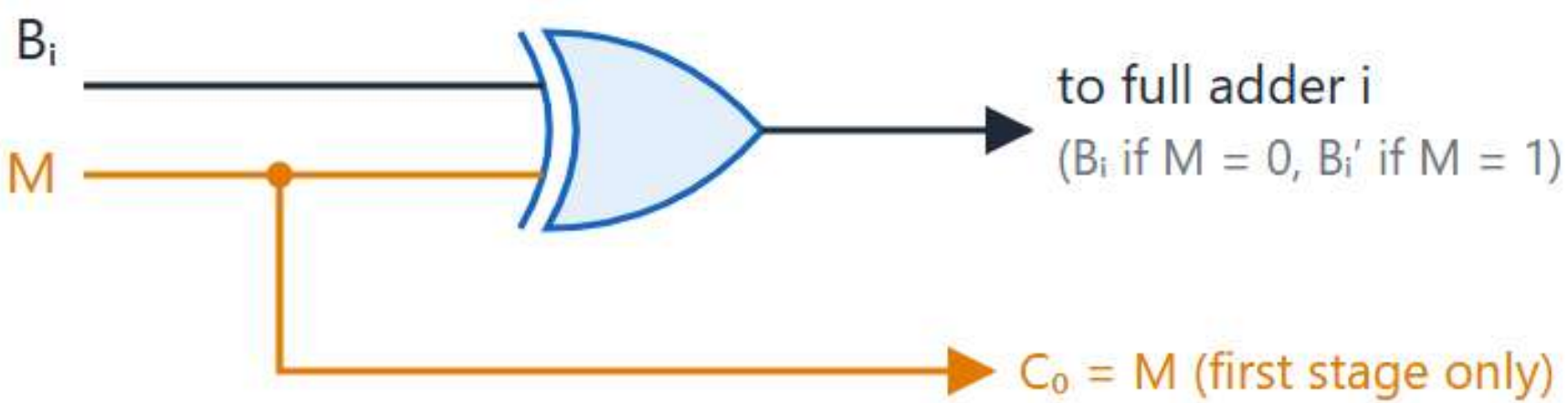


A full subtractor built from two half subtractors and one OR gate — HS_1 computes A−B, HS_2 then subtracts B_{in} from that result.

Combined adder–subtractor

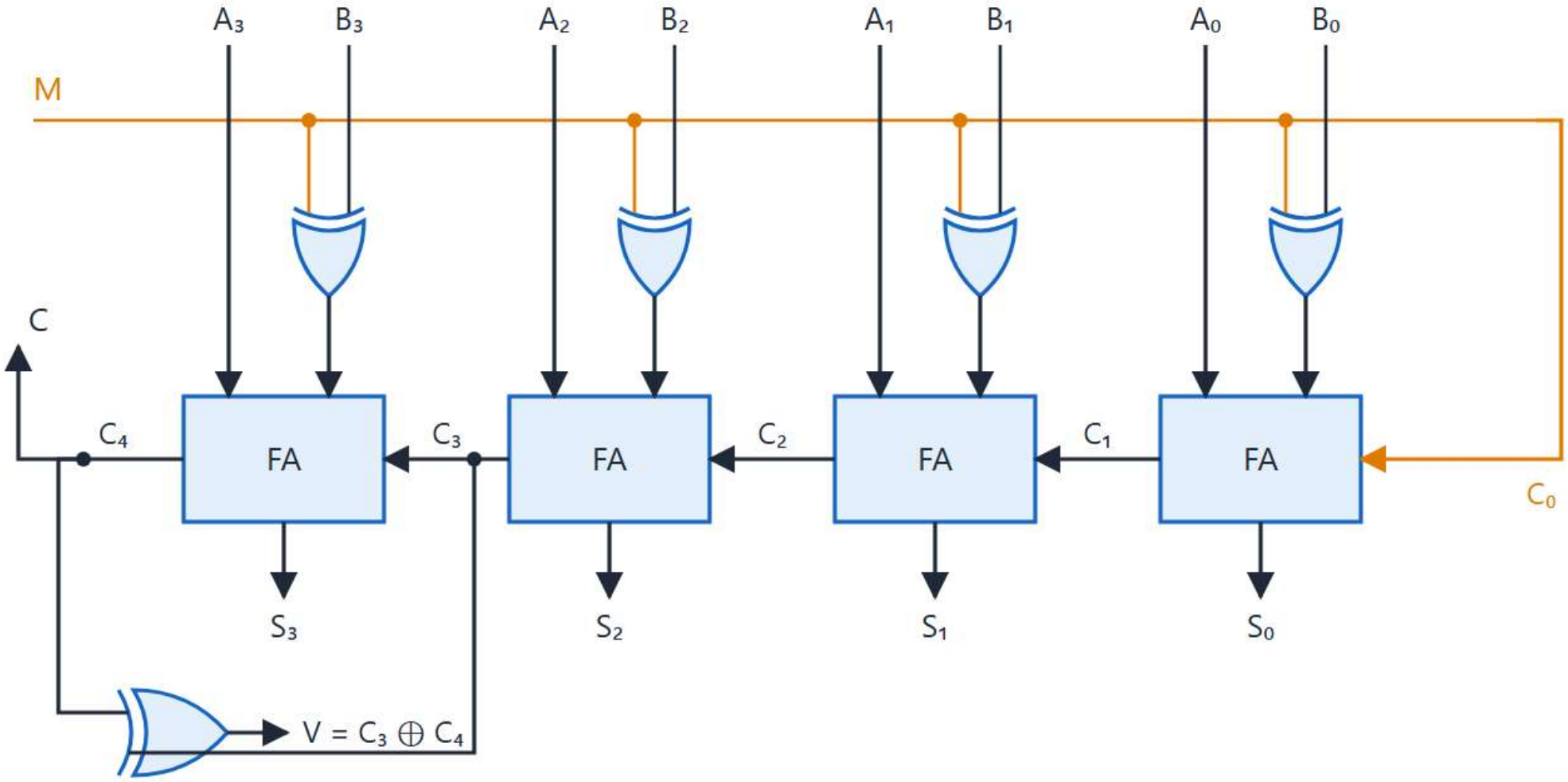
- 2’s complement turns subtraction into addition: **A – B = A + B′ + 1**
- One line **M** (0 = add, 1 = subtract) controls both operations
- One **XOR gate** in front of every bit of B; M also feeds the first stage’s carry-in
- **M = 0:** B passes unchanged, carry-in 0: computes A + B
- **M = 1:** B is inverted, carry-in 1: computes A + B′ + 1 = A − B
- Overflow (signed): $V = C_3 \oplus C_4$; the final carry matters only for unsigned numbers

One XOR gate per bit of B



One XOR gate per bit of B — the same M line that flips this gate also becomes the first stage's carry-in.

Four-bit adder-subtractor



Four-bit adder-subtractor: M = 0 adds, M = 1 subtracts; $V = C_3 \oplus C_4$ flags overflow. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.13.

Worked example: A = 1001 (9), B = 0101 (5)


```
M = 0 (add):        1001 + 0101 = 01110 = 14    (9 + 5 ✓)
M = 1 (subtract):   1001 + 1010 + 1 = 10100
                    drop the final carry → 0100 = 4    (9 − 5 ✓)
```

- B′ = 1010 when M = 1, and the carry-in to bit 0 is 1
- The final carry-out is discarded: it is expected when A ≥ B

2.6 Decimal (BCD) Adder

IN SIMPLE TERMS

Add the two digits in binary; if the result is more than 9 (or a carry came out), add 6 to skip the six unused codes and pass a 1 to the next digit.

The correction rule

- Add two BCD digits (plus any carry-in) in a 4-bit binary adder: raw sum **Z** (0–19) and carry **K**
- If **Z ≤ 9 and K = 0**: already valid BCD; use it as is
- If **Z ≥ 10, or K = 1**: add **0110 (6)**; the correction's decade carry goes to the next decade
- $Z \geq 10$ is detected by $Z_8Z_4 + Z_8Z_2$
- **Output carry = $K + Z_8Z_4 + Z_8Z_2$**: 1 means "add 6" and is also the carry into the next decade

Binary sum and BCD sum (1 of 2)

Up to 9 the raw sum and the BCD digit are identical: no correction needed.

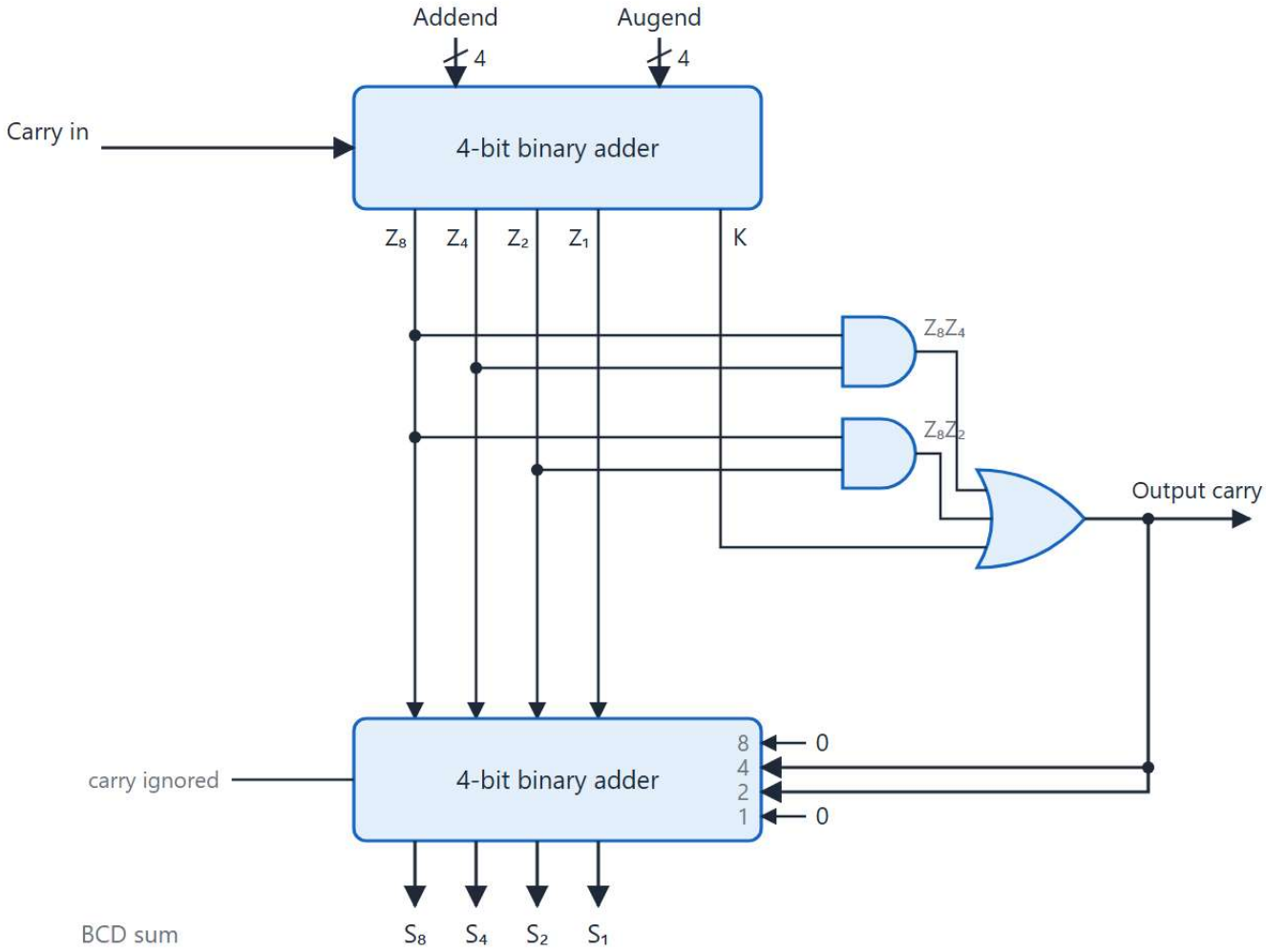
Decimal	K	$Z_8Z_4Z_2Z_1$	Output carry	$S_8S_4S_2S_1$
0	0	0000	0	0000
1	0	0001	0	0001
2	0	0010	0	0010
3	0	0011	0	0011
4	0	0100	0	0100
5	0	0101	0	0101
6	0	0110	0	0110
7	0	0111	0	0111
8	0	1000	0	1000
9	0	1001	0	1001

Binary sum and BCD sum (2 of 2)

From 10 onward the two diverge; adding 0110 brings S back into range every time.

Decimal	K	$Z_8Z_4Z_2Z_1$	Output carry	$S_8S_4S_2S_1$
10	0	1010	1	0000
11	0	1011	1	0001
12	0	1100	1	0010
13	0	1101	1	0011
14	0	1110	1	0100
15	0	1111	1	0101
16	1	0000	1	0110
17	1	0001	1	0111
18	1	0010	1	1000
19	1	0011	1	1001

Block diagram of a BCD adder



Block diagram of a BCD adder. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.14.

Worked example 1: 8 + 7 (Z ≥ 10 case)

A = 1000 (8), B = 0111 (7), C_{in} = 0
Binary adder: 1000 + 0111 = 1111 → Z = 1111 (15), K = 0
Z_8Z_4 = 1 → Output carry = 0 + 1 + 1 = 1 (add 6)
Correction adder: 1111 + 0110 = 1 0101
S = 0101 (5), decade carry = 1

- Result: “1” and “5”, which is 15 in BCD; check 8 + 7 = 15 ✓

Worked example 2: 9 + 9 (K = 1 case)

A = 1001 (9), B = 1001 (9), C_{in} = 0
Binary adder: 1001 + 1001 = 1 0010 → Z = 0010 (2), K = 1
Z_8Z_4 = 0, Z_8Z_2 = 0, but K = 1 → Output carry = 1 (add 6)
Correction adder: 0010 + 0110 = 1000
S = 1000 (8), decade carry = 1

- Result: “1” and “8”, which is 18 in BCD; check 9 + 9 = 18 ✓
- Z alone (0010) looks small: only checking K catches it

2.7 Binary Multiplier

IN SIMPLE TERMS

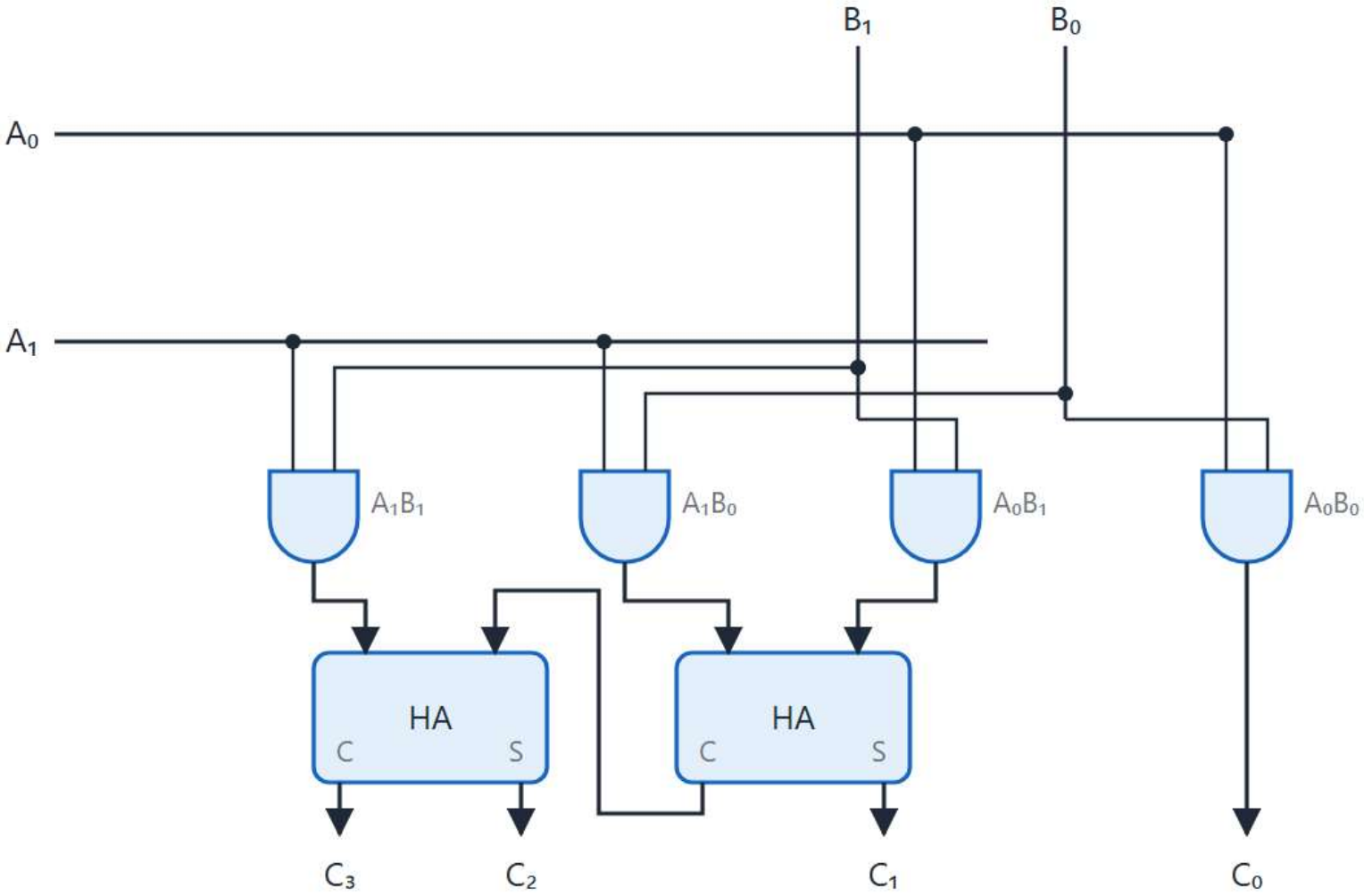
Multiplying bits is just AND; the partial products are then added column by column with half and full adders.

2-bit × 2-bit multiplier: A = 10 (2), B = 11 (3)


```
          A1    A0
    ×     B1    B0
  ----------------
       A1B0  A0B0     ← PP0 = A·B0
 A1B1  A0B1           ← PP1 = A·B1, shifted left 1
  P3    P2    P1   P0   = 0 1 1 0 = 6 ✓
```

- Each partial product is all zeros (multiplier bit 0) or a copy of the multiplicand (bit 1)
- **Each partial product bit is the AND** of a multiplicand bit and a multiplier bit
- m-bit × n-bit: product up to m + n bits; the AND array needs m × n gates
- Half adders where 2 bits share a column; full adders once 3 or more do

Two-bit by two-bit binary multiplier



Two-bit by two-bit binary multiplier. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.15.

2.8 Magnitude Comparator

IN SIMPLE TERMS

Compare from the most significant bit down: the first bit position where the numbers differ decides which is larger.

1-bit comparator

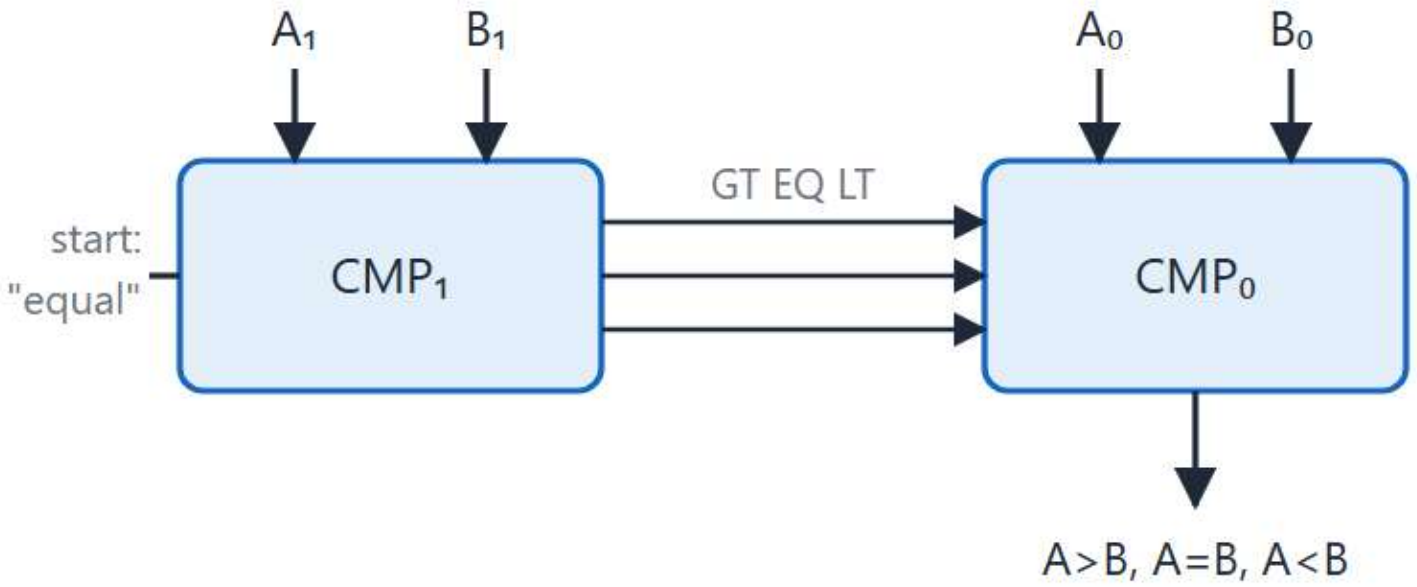
(A>B) = AB′ (A<B) = A′B (A=B) = A′B′ + AB = (A ⊕ B)′ (the XNOR of A and B)

A	B	A>B	A=B	A<B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Cascading to n bits

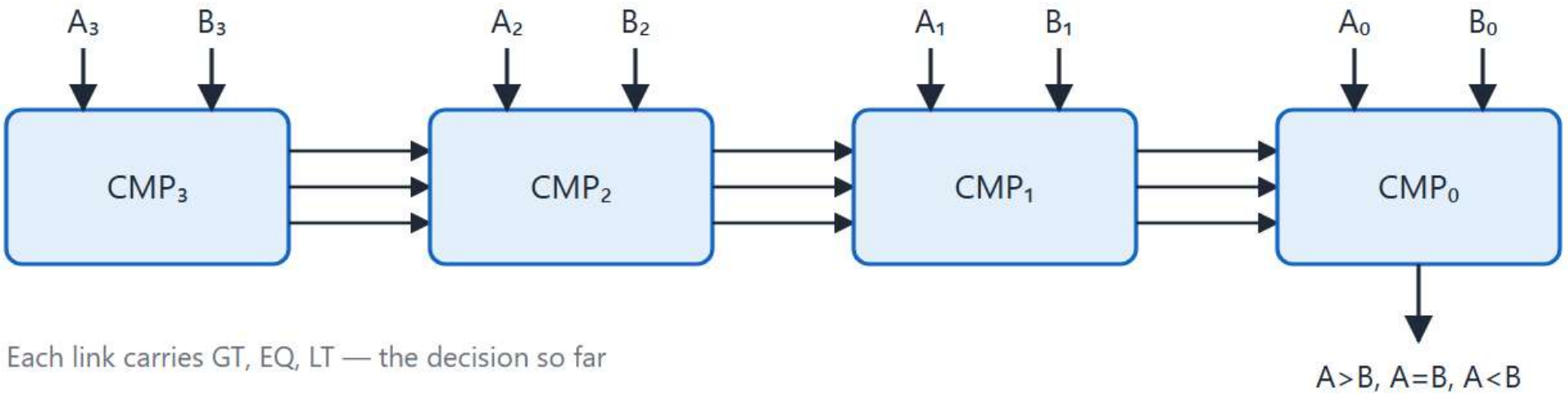
- Comparison starts at the **most significant bit**: if the MSBs differ, lower bits cannot change the outcome
- Only when a pair of bits is **equal** does the comparison continue to the next bit
- With $x_i = (A_i = B_i)$, for 2 bits: $(A>B) = A_1B_1' + x_1A_0B_0'$
- $(A<B) = A_1'B_1 + x_1A_0'B_0$ $(A=B) = x_1x_0$
- Each stage passes a running (GT, EQ, LT) decision to the next; the first cascade input is (0, 1, 0), “equal so far”
- Only the last stage’s outputs are the overall result

Two-bit comparison as two cascaded stages



Two-bit comparison as two cascaded 1-bit stages, most significant bit first.

Four-bit comparison as a chain



Four-bit comparison as a chain of 1-bit stages, each passing GT, EQ, LT to the next.

Four-bit comparator: gate-level equations

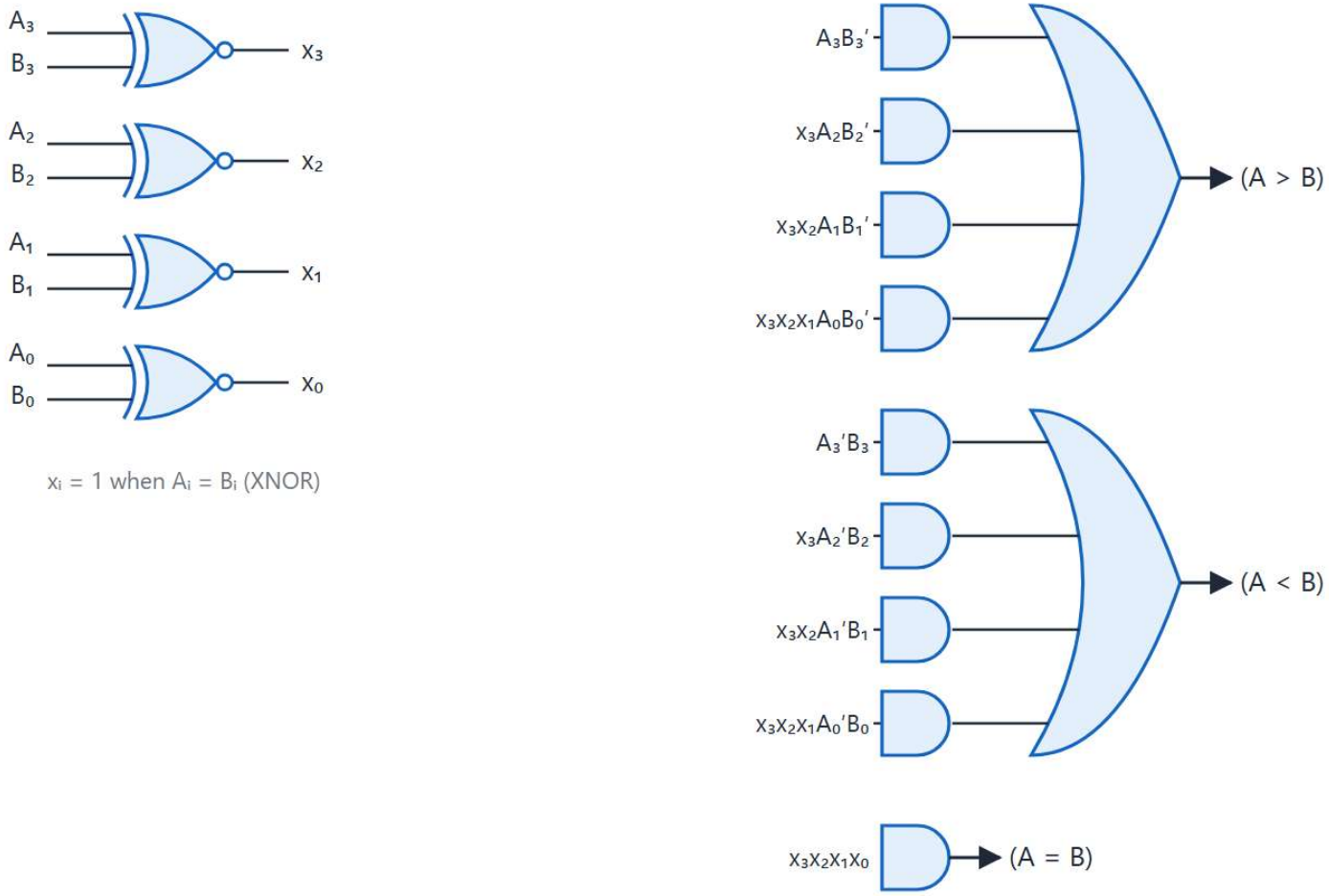
$x_i = A_iB_i + A_i'B_i'$ (XNOR: 1 when bit i of A and B are equal)

$(A = B) = x_3x_2x_1x_0$

$(A > B) = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$

- (A < B) is the same as (A > B) with A and B swapped in each term

Four-bit magnitude comparator



Four-bit magnitude comparator. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.17.

Worked example: A = 1011 (11), B = 1010 (10)


```
Bit 3: A3=1, B3=1 → equal, still equal to bit 2
Bit 2: A2=0, B2=0 → equal, still equal to bit 1
Bit 1: A1=1, B1=1 → equal, still equal to bit 0
Bit 0: A0=1, B0=0 → A0 > B0, every higher bit tied
Result: A > B   (11 > 10) ✓
```

- Decided entirely by the least significant bit, because every bit above it matched

2.9 Decoders

IN SIMPLE TERMS

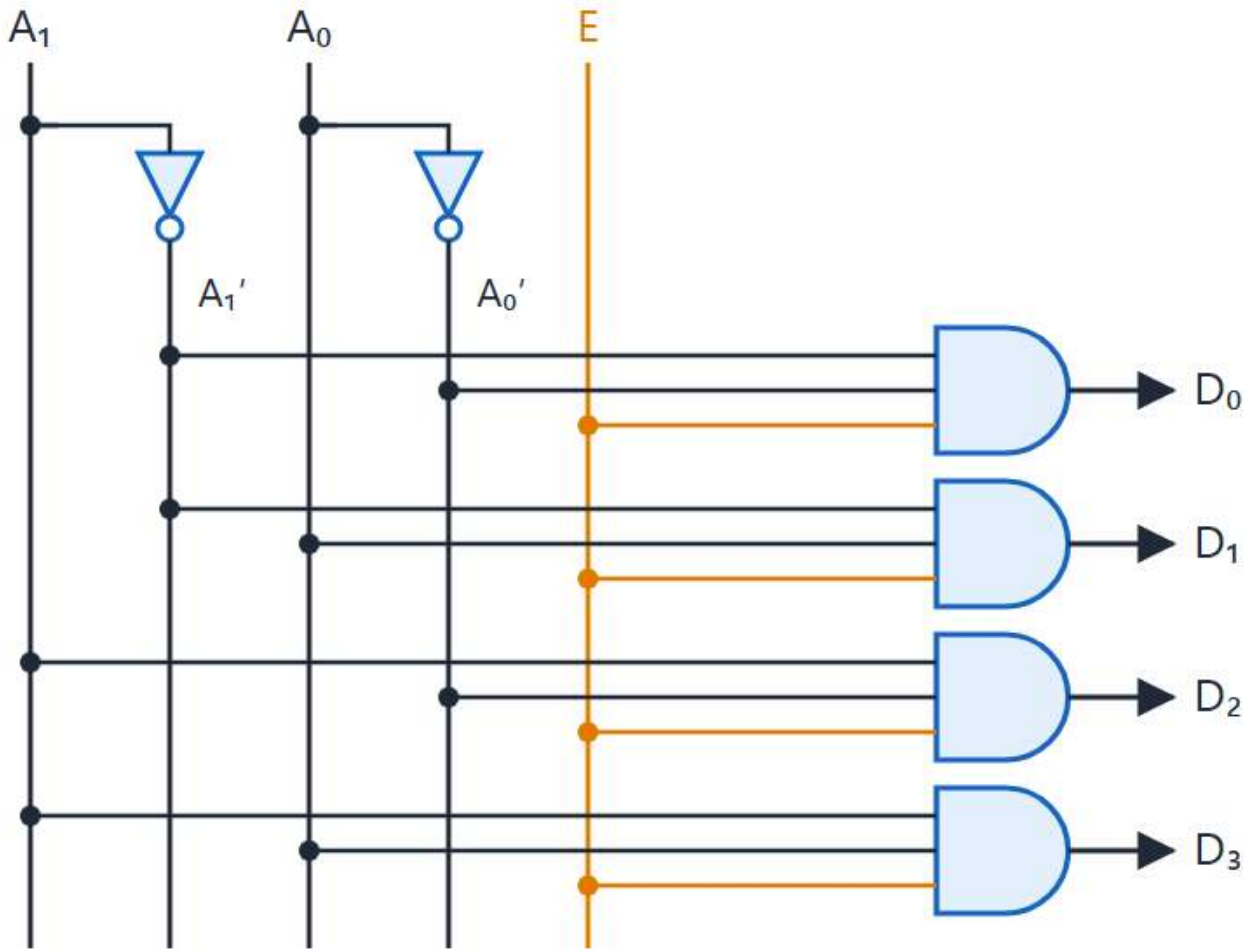
A decoder turns a binary number into “one wire high”: input 101 lights up output 5 and nothing else.

2-to-4 decoder

$D_0 = A_1'A_0'$ $D_1 = A_1'A_0$ $D_2 = A_1A_0'$ $D_3 = A_1A_0$. A decoder is a physical implementation of minterm generation: each output **is** one minterm.

A_1	A_0	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

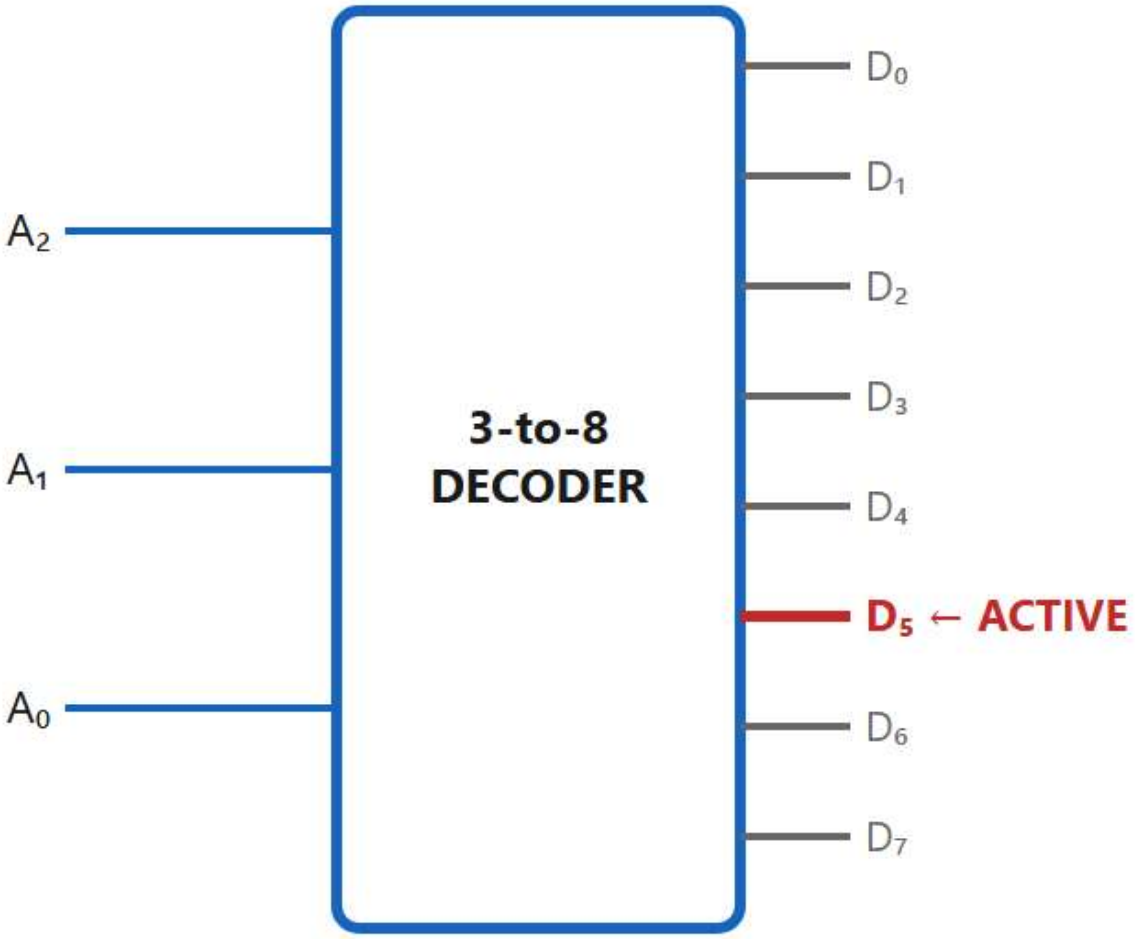
2-to-4 decoder with enable



- Each output AND gate also receives E
- E = 0 forces every output to 0
- E = 1: the address inputs select one output
- This makes a decoder useful as chip-select logic

2-to-4 decoder with enable: each output AND gate also receives E, so all outputs are 0 when E = 0.

3-to-8 decoder



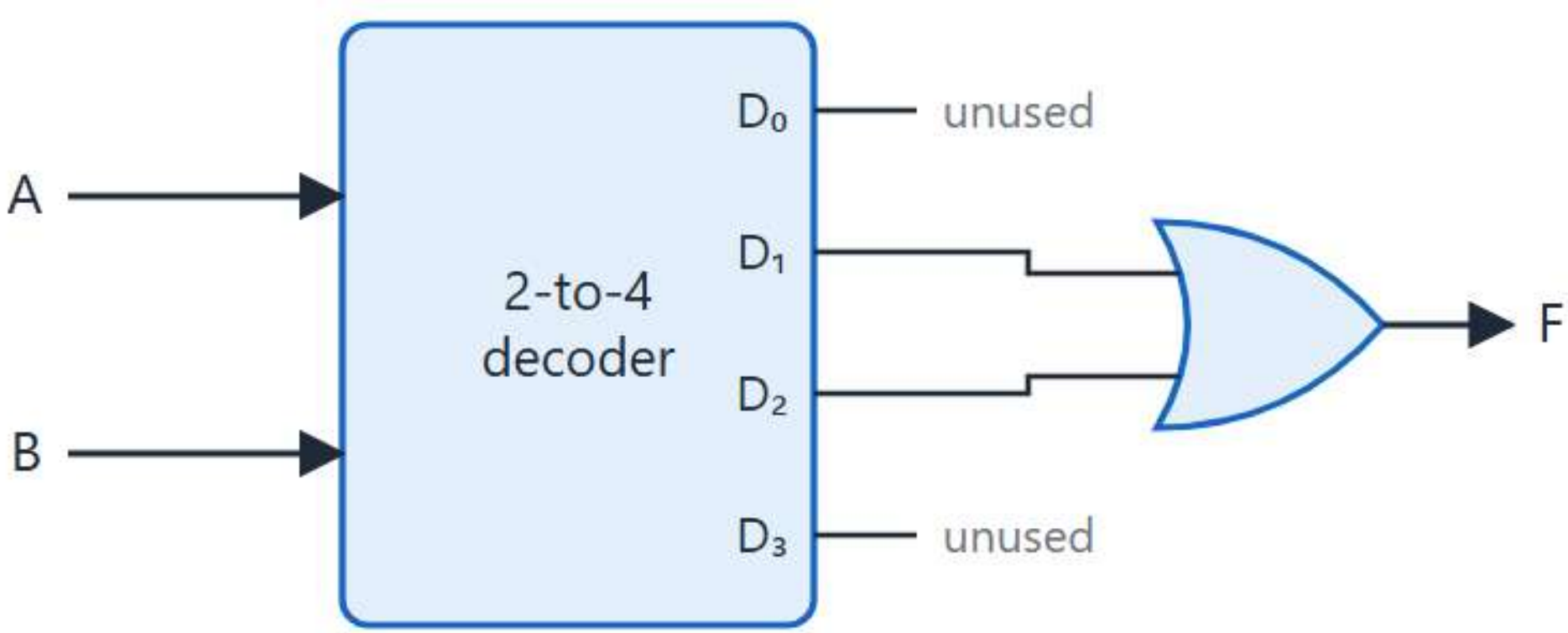
- Inputs A_2 A_1 A_0 activate exactly one of the outputs D_0–D_7
- Input 101 (5) activates only D_5
- Every other output stays at 0
- Each memory chip or bank is wired to one output, so only one is selected

A 3-to-8 decoder selects exactly one of eight outputs — here, input 101 (5) activates only D_5.

Enable and implementing functions

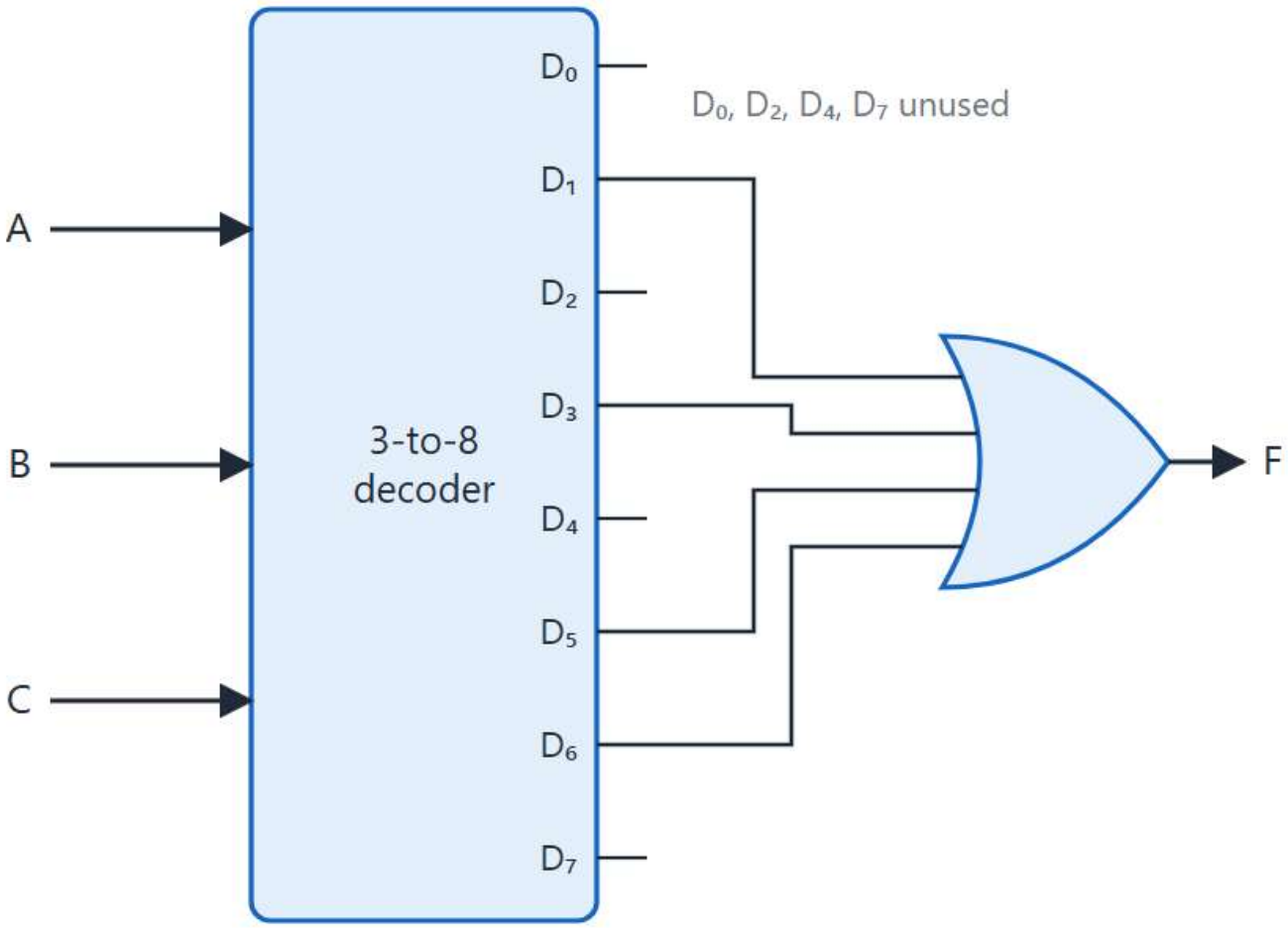
- An **enable** input E: when E = 0 every output is 0, so only the addressed chip's select line goes active
- E = 0 silences the whole decoder; it does not choose a default output
- Every output is already a minterm, so any function is: feed n variables into an n-to-2^n decoder and **OR** the outputs for its minterms
- Example: $F(A,B) = \Sigma m(1,2) = D_1 + D_2$: one 2-input OR gate
- No AND gates and no K-map work are needed

F(A,B) = Σm(1,2) from a decoder



F(A,B) = Σm(1,2) = $D_1 + D_2$ from a 2-to-4 decoder and one OR gate.

F(A,B,C) = Σm(1,3,5,6)



- $F = D_1 + D_3 + D_5 + D_6$
- One four-input OR gate combines the four chosen outputs
- No K-map simplification is needed
- The price is a full decoder, which uses more gates than a minimised circuit

F(A,B,C) = Σm(1,3,5,6): decoder outputs D_1, D_3, D_5 and D_6 feed one four-input OR gate.

2.10 Encoders

IN SIMPLE TERMS

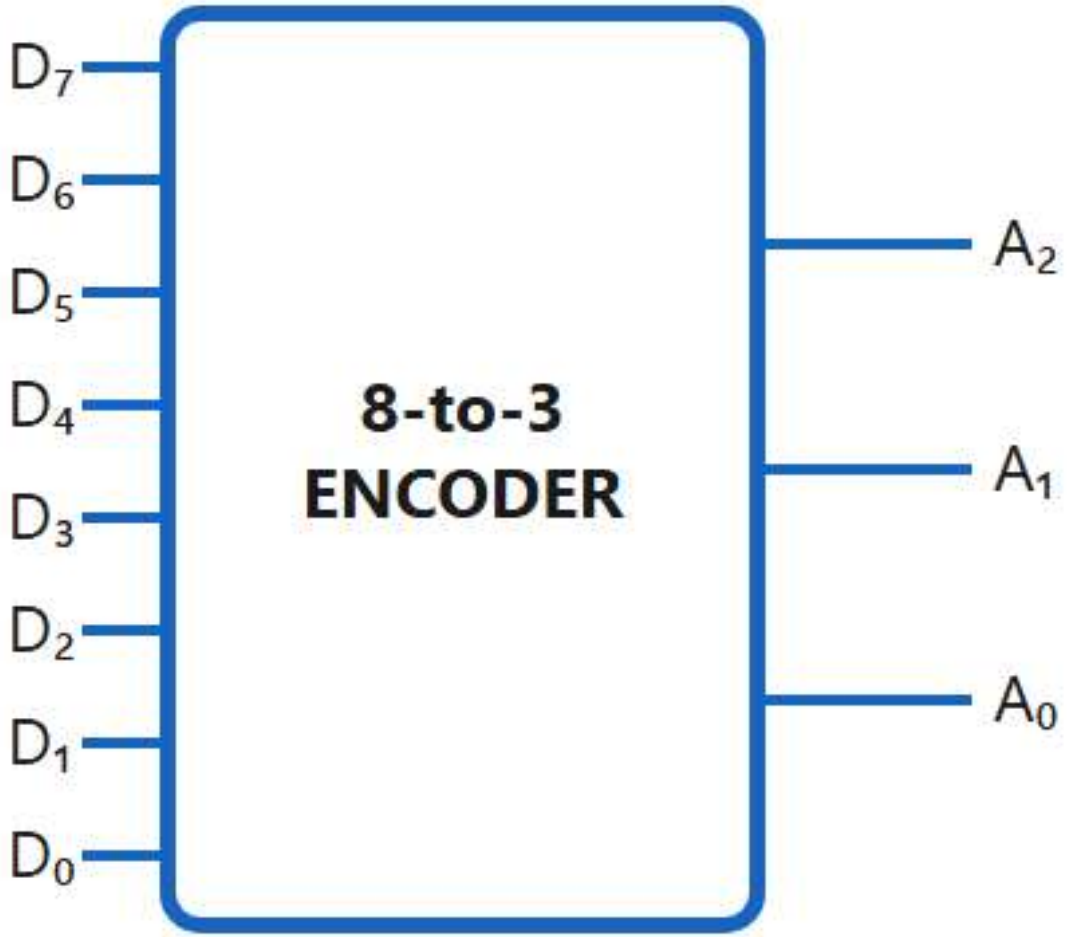
An encoder does the reverse: it reports which input wire is high as a binary number; a priority encoder picks the highest one if several are high.

8-to-3 (octal-to-binary) encoder


```
A0 = D1 + D3 + D5 + D7     (inputs whose index has bit 0 set)
A1 = D2 + D3 + D6 + D7     (inputs whose index has bit 1 set)
A2 = D4 + D5 + D6 + D7     (inputs whose index has bit 2 set)
```

- Each output is an OR of the four inputs whose index has that bit set: three 4-input OR gates, no AND gates, no K-maps
- Example: only D_5 active → $A_2 = 1$, $A_1 = 0$, $A_0 = 1$ → $A_2A_1A_0 = 101$ ✓

Octal-to-binary encoder



Octal-to-binary encoder

The priority problem and priority encoder

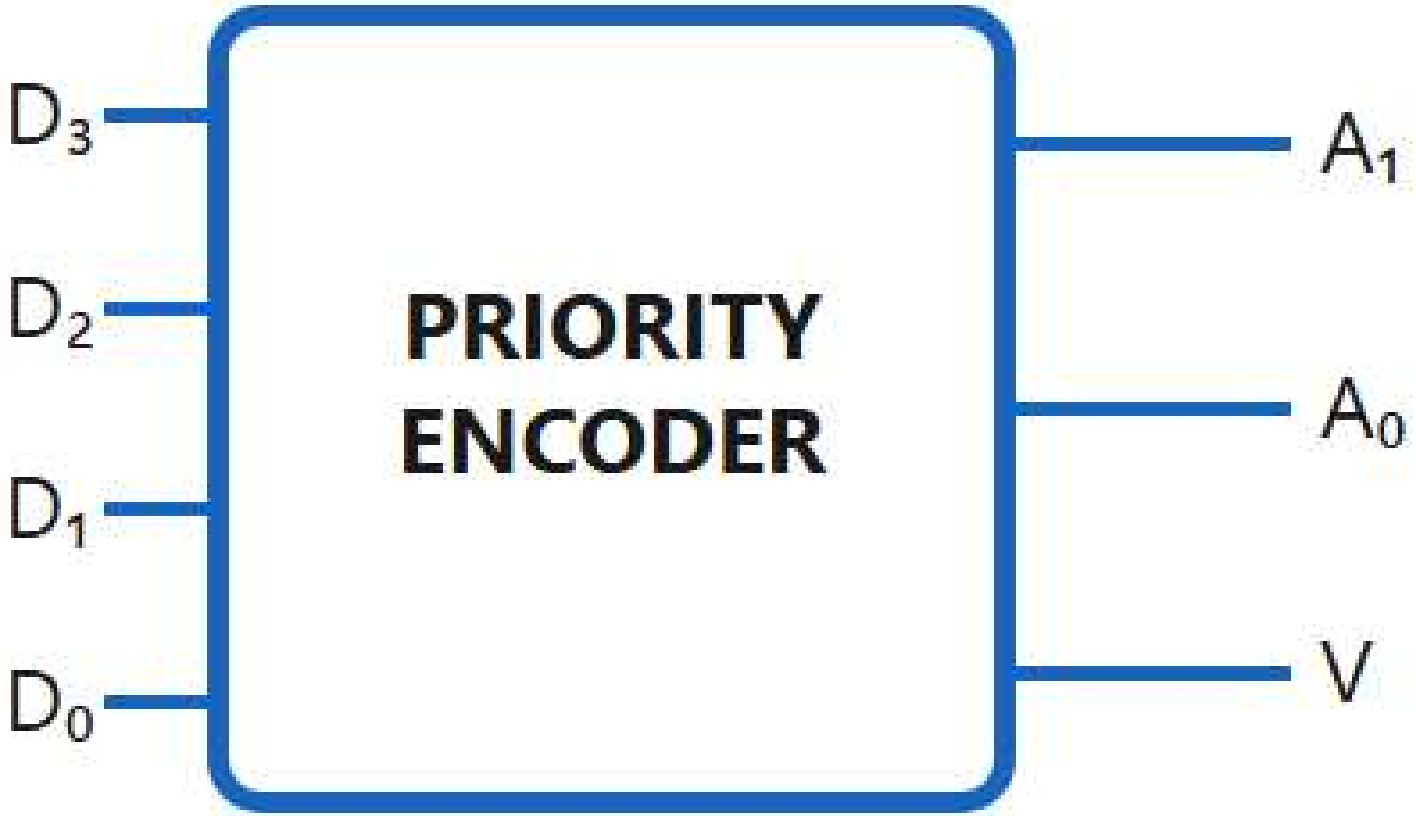
- A plain encoder assumes exactly **one** input is active; with two or more, the outputs become the OR of several codes: garbage
- A **priority encoder** outputs the code of the **highest-priority** input: conventionally the highest index
- **V** is a valid-output flag: 0 only when no input is active, so $A_1 = A_0 = 0$ is not ambiguous
- Example: $D_2 = D_1 = 1$ ($D_3 = D_0 = 0$) → output **10 (2)**; D_1 is ignored
- Highest index wins, not lowest: always check the convention of a chip or textbook

4-input priority encoder

X means don't care: once D_3 is 1, D_2, D_1, D_0 do not matter.

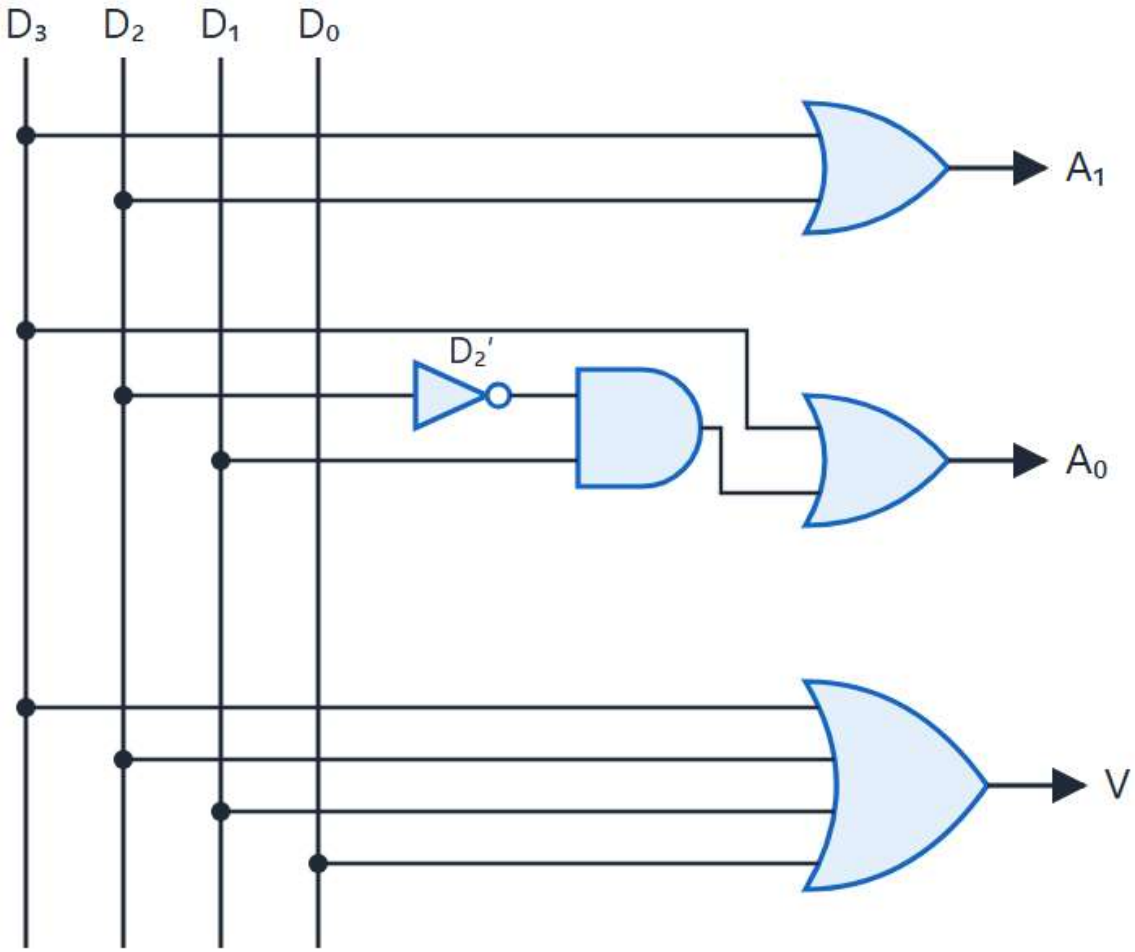
D_3	D_2	D_1	D_0	A_1	A_0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

4-input priority encoder block



4-input priority encoder block

Four-input priority encoder



Four-input priority encoder: $A_1 = D_3 + D_2$, $A_0 = D_3 + D_2'D_1$, $V = D_3 + D_2 + D_1 + D_0$. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.23.

2.11 Multiplexers

IN SIMPLE TERMS

A multiplexer is a selector switch: the select lines choose which one of several inputs is passed to the single output.

What a multiplexer does

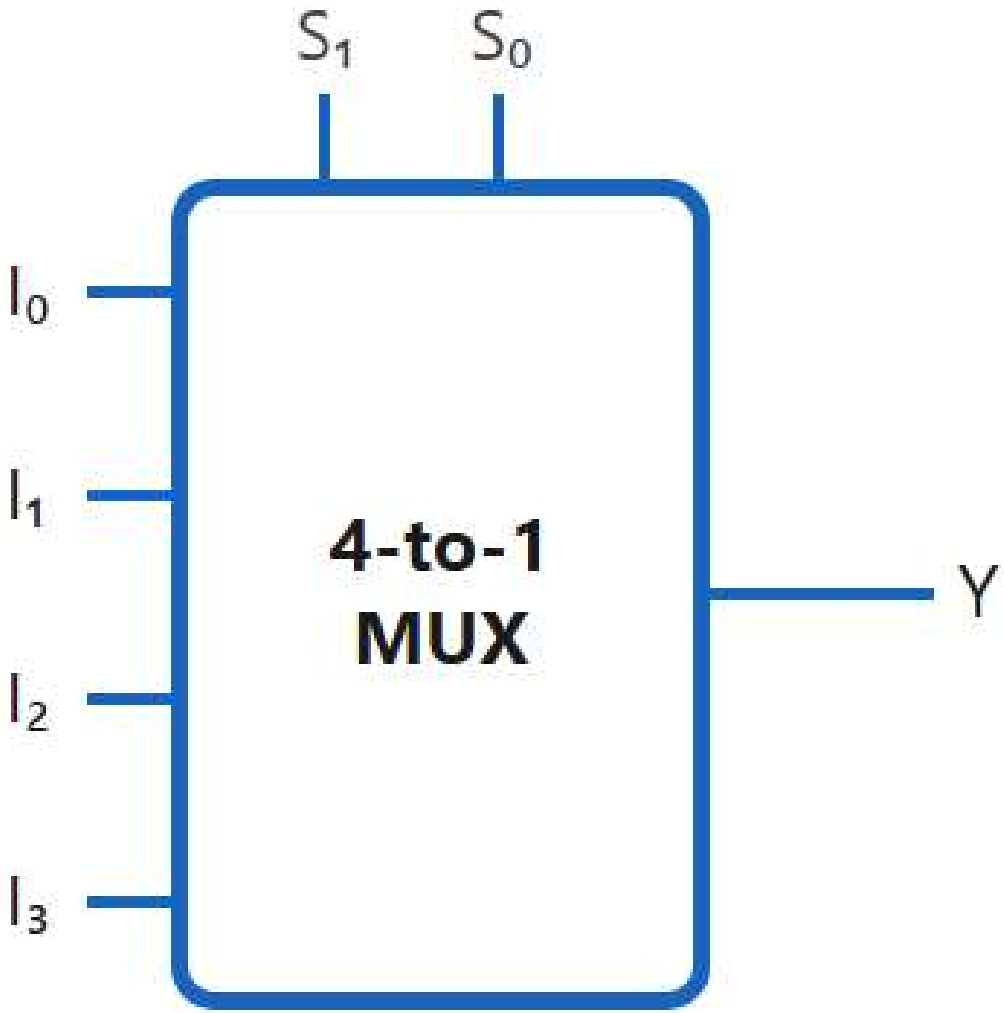
- A **multiplexer (MUX)** selects one of 2^n input lines and routes it to a single output, using n **select lines**
- The select lines act as an address
- A 2-to-1 MUX has 1 select line; a 4-to-1 MUX has 2
- An n-select-line MUX handles exactly 2^n inputs: never more, never fewer

4-to-1 multiplexer

$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$. Only one AND term is enabled at a time, so Y equals exactly one data input.

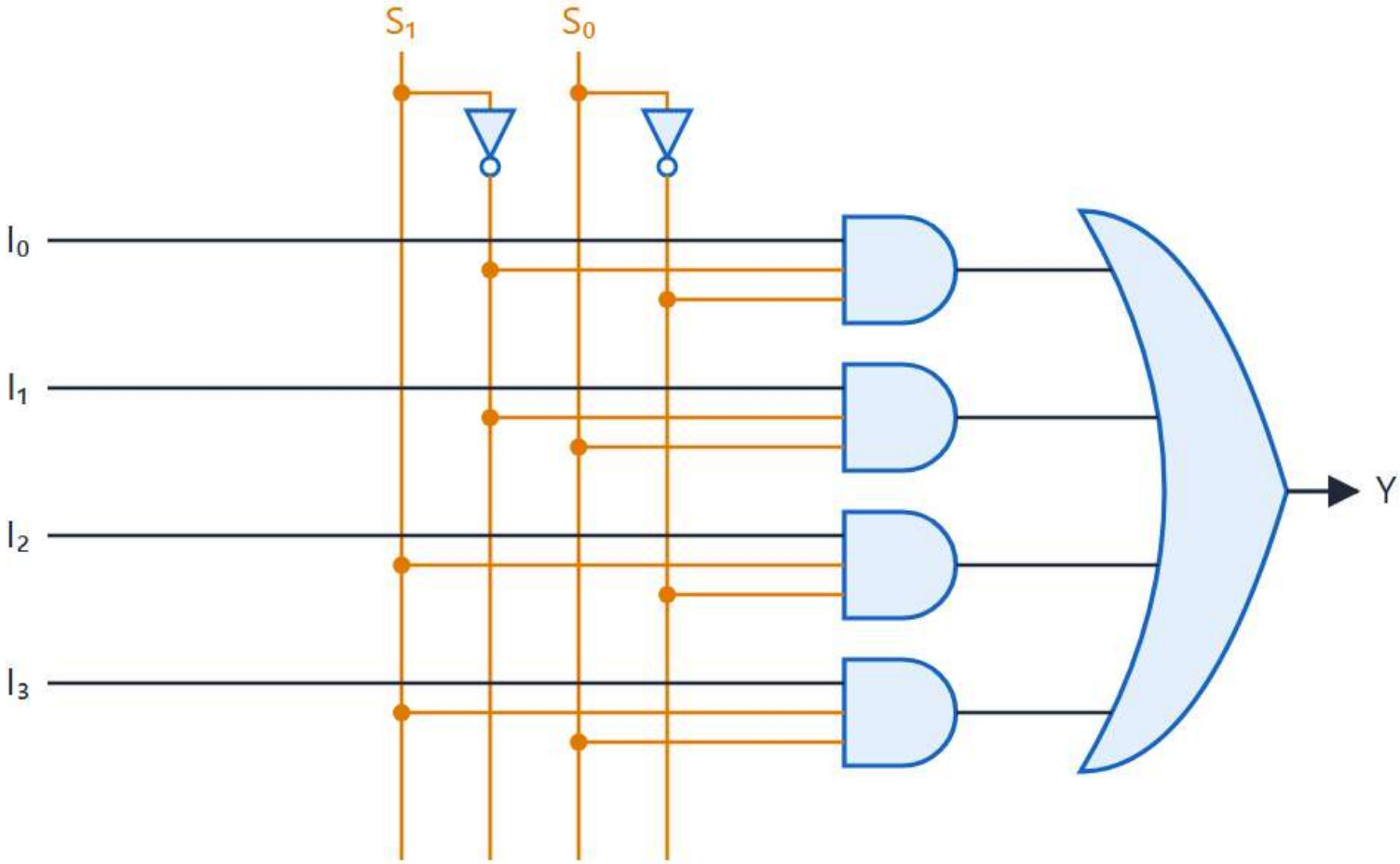
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

4-to-1 MUX



A 4-to-1 MUX — the 2-bit select address S_1S_0 (entering the top) picks which of I_0–I_3 reaches the output Y.

Four-to-one-line multiplexer

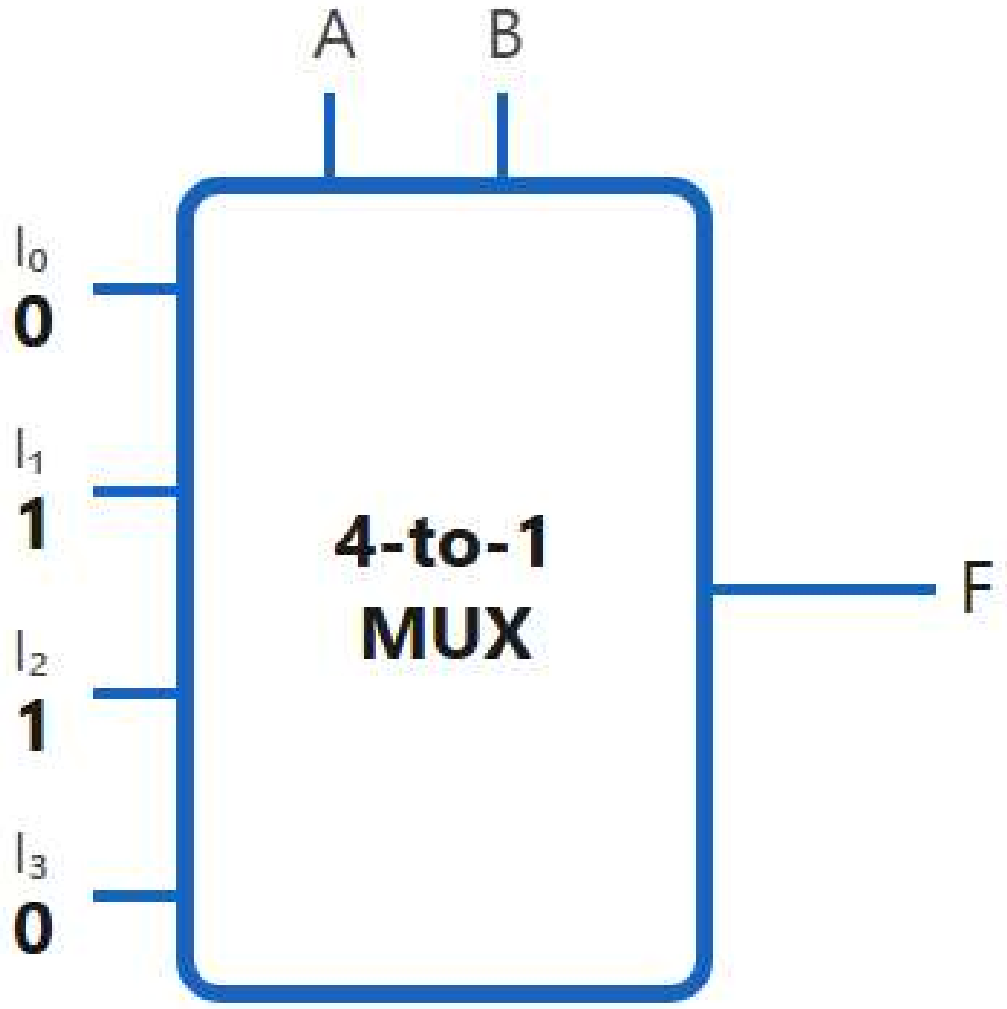


Four-to-one-line multiplexer. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 4.25.

Implementing Boolean functions with a MUX

- The MUX equation has the shape of a **sum of minterms**: each term is a minterm of the select lines times a data input
- Tie each data input to 0 or 1 to match a truth table, and the MUX **is** the function
- An n-variable function needs a 2^n-to-1 MUX and no other gates
- **Example: XOR** with $S_1 = A$, $S_0 = B$: $I_0 = 0$, $I_1 = 1$, $I_2 = 1$, $I_3 = 0$
- Check: A = 1, B = 0 selects $I_2 = 1$, and $1 \oplus 0 = 1$ ✓

XOR from a MUX

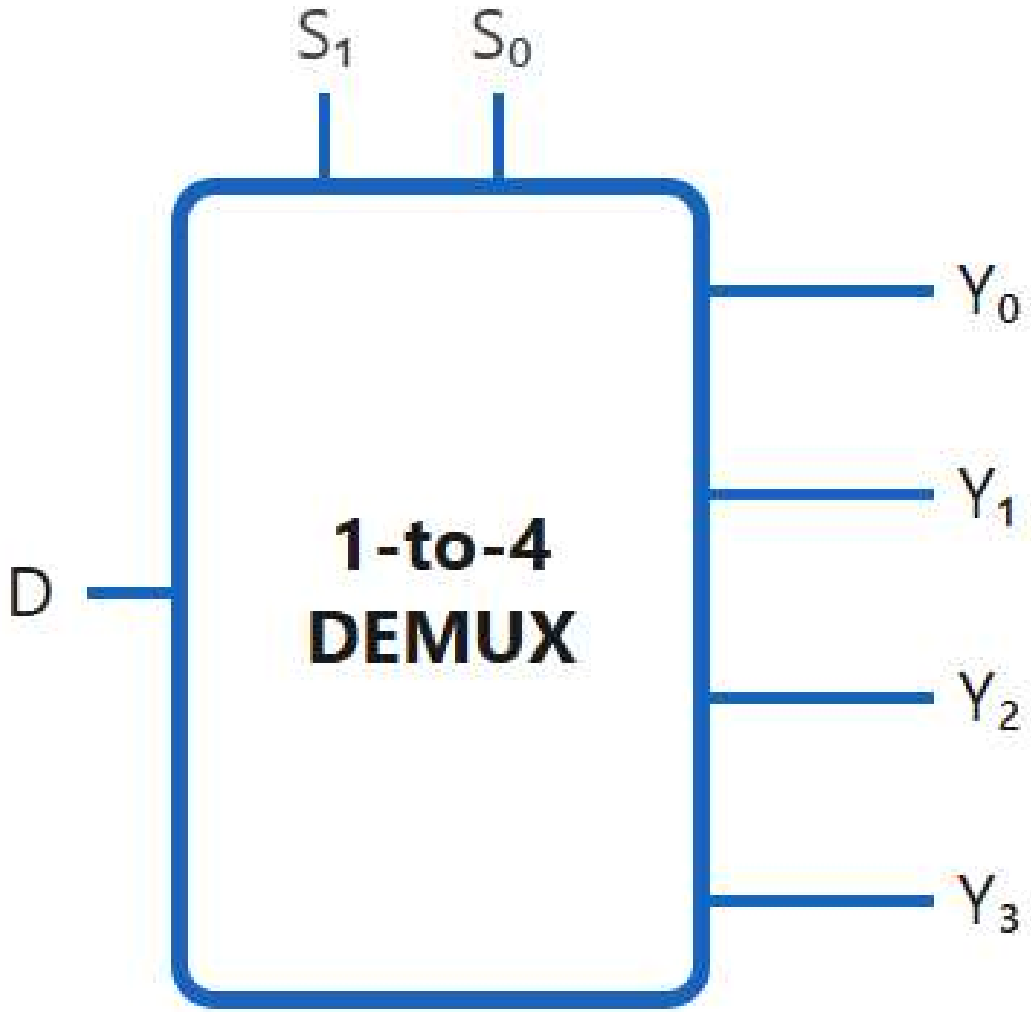


Tying constants to the four data inputs, with A and B as the select lines, turns the MUX itself into an XOR circuit — no AND, OR, or XOR gates anywhere.

Demultiplexer: the inverse operation

- A **DEMUX** routes one data input D to one of 2^n outputs, chosen by n select lines
- All other outputs stay 0
- 1-to-4 DEMUX: $Y_0 = S_1'S_0'D$ $Y_1 = S_1'S_0D$ $Y_2 = S_1S_0'D$ $Y_3 = S_1S_0D$
- A MUX answers “which input do I read?”; a DEMUX answers “which destination gets this value?”
- A demultiplexer **is** a decoder: D takes the place of the shared enable input

1-to-4 DEMUX



A 1-to-4 DEMUX — the select address S_1S_0 picks which single output D is routed to; every other output stays 0.

2.12 Sequential Circuits

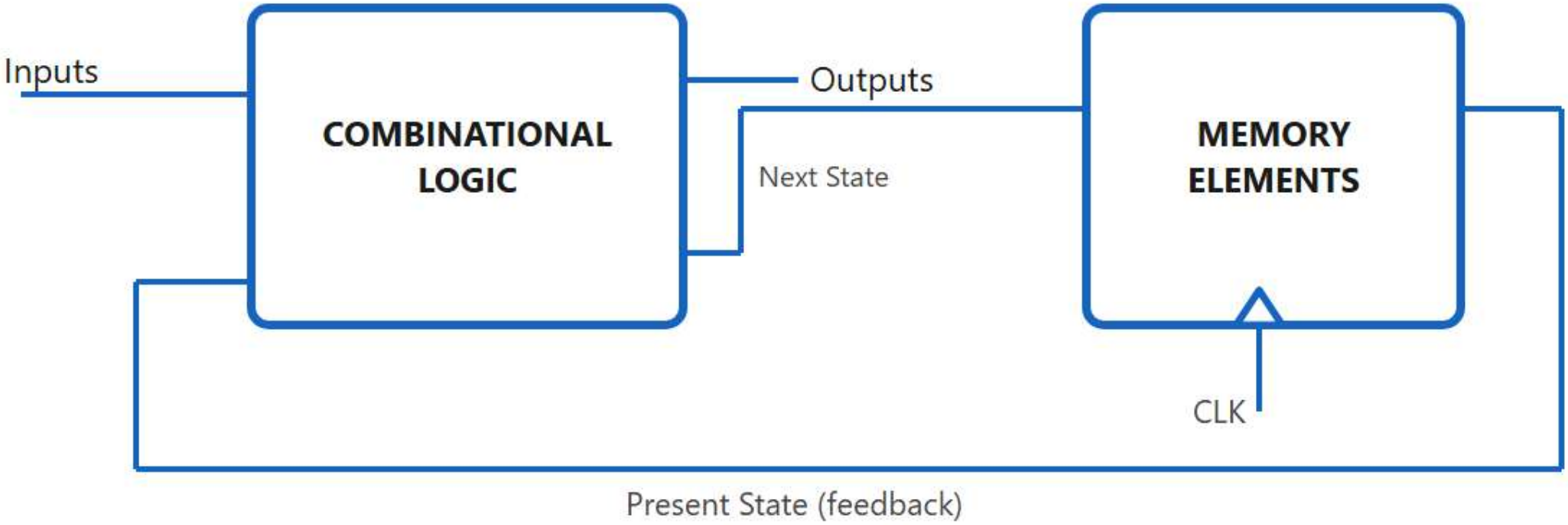
IN SIMPLE TERMS

A sequential circuit remembers: its output depends on what happened before (its state) as well as on the inputs now.

Sequential circuits

- Output depends on the present **inputs** and the current **state**: stored information about what happened before
- State comes from feeding part of the output back through storage elements (latches or flip-flops)
- **Next State = f(Inputs, Present State)**
- A combinational block computes the outputs and the next state; memory elements store the next state and feed it back as the present state
- Remove the present-state feedback and the circuit collapses into a combinational one
- The smallest sequential circuit, the latch, stores one bit with gates and feedback alone

General shape of a sequential circuit



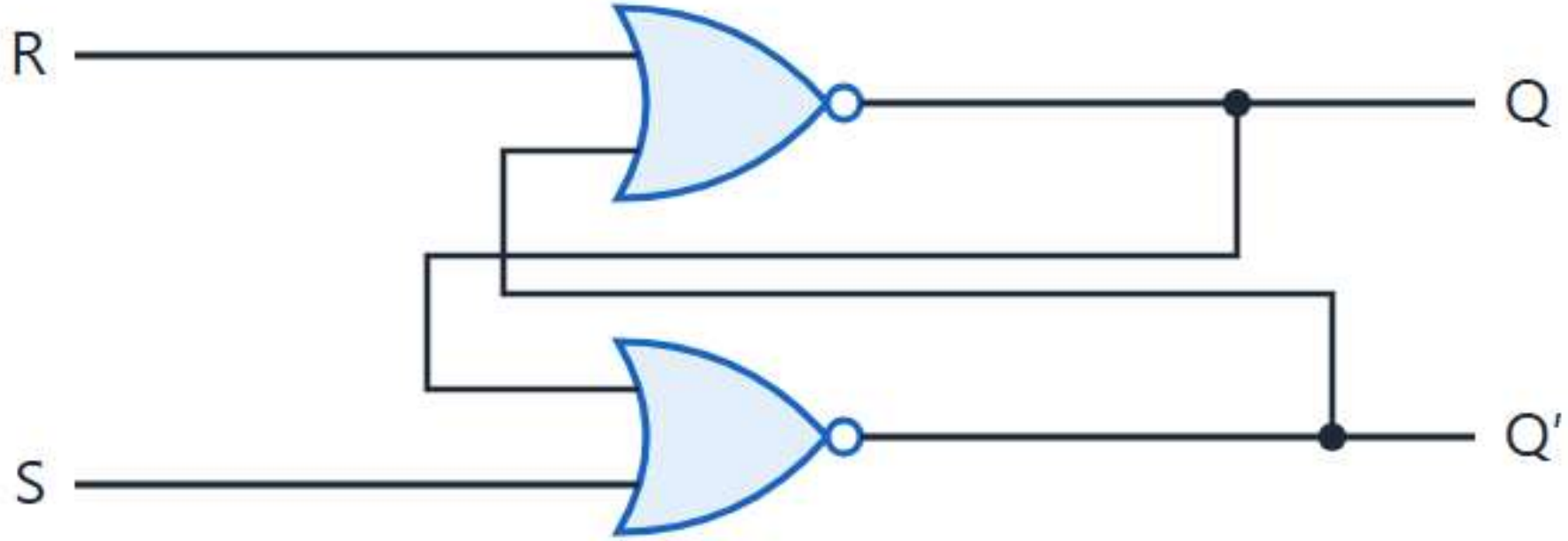
The general shape of every sequential circuit — outputs and the next state both come from the combinational block, and the present state loops back from memory to influence the next computation.

2.13 Latches

IN SIMPLE TERMS

A latch is two gates feeding each other, so it holds a 1 or a 0 by itself; Set and Reset change what it holds.

SR latch with cross-coupled NOR gates



SR latch with two cross-coupled NOR gates.

SR latch

Q = NOR(R, Q’) Q’ = NOR(S, Q). S = R = 1 forces both Q and Q’ to 0, breaking the complement relationship the latch depends on: always avoid it.

S	R	Q_{next}	State
0	0	Q	**Hold**
1	0	1	**Set**
0	1	0	**Reset**
1	1	Invalid	**Forbidden**

Worked example: tracing a sequence (start Q = 0)

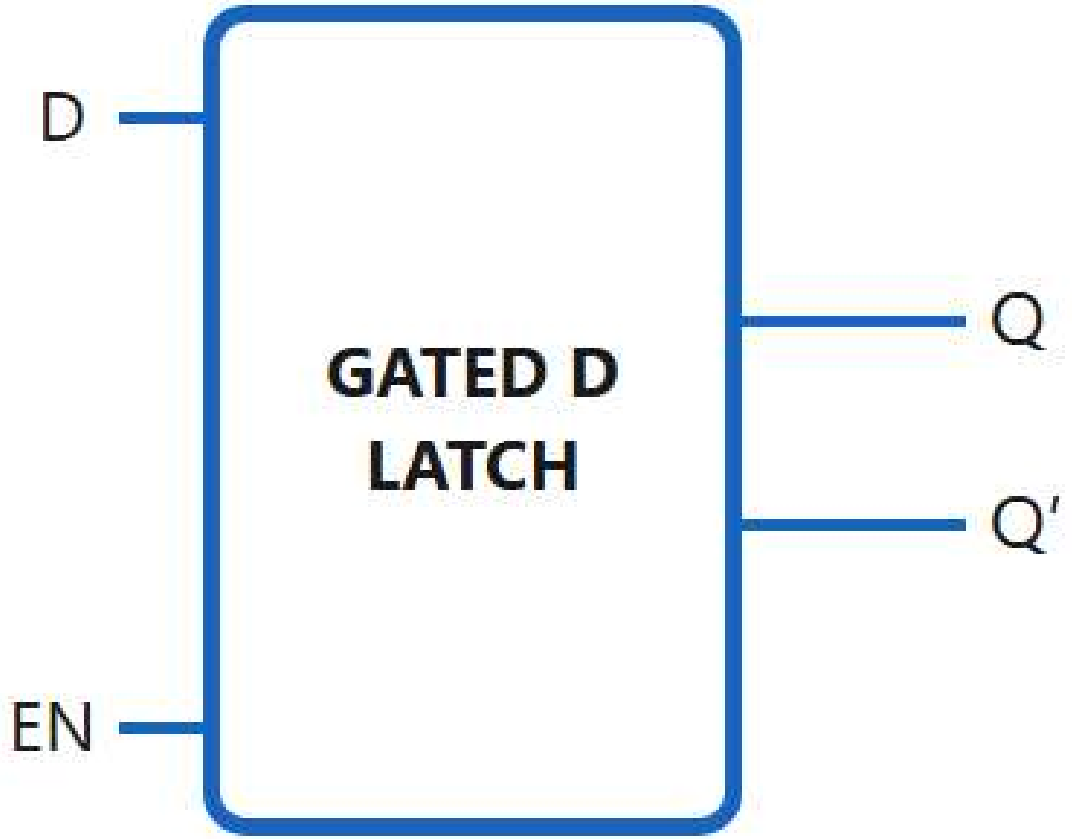

```
Set   (S=1, R=0):  Q becomes 1
Hold  (S=0, R=0):  Q stays 1
Reset (S=0, R=1):  Q returns to 0
```

- In Hold the output stays at its last value while both inputs are 0
- That hold-in-between behaviour is the whole point: it is memory

The gated D latch

- Drive **S = D** and **R = D’**: S and R are always complements, so the forbidden state cannot happen **by construction**
- An **Enable (EN)** input, through two AND gates gating D and D’, controls when the latch updates
- While EN = 1, Q follows D continuously: the latch is “transparent”
- When EN drops to 0, Q freezes at whatever D was last
- This is **level-sensitive** behaviour; a flip-flop (next topic) adds edge triggering on top

Gated D latch



Gated D latch

2.14 Flip-Flops

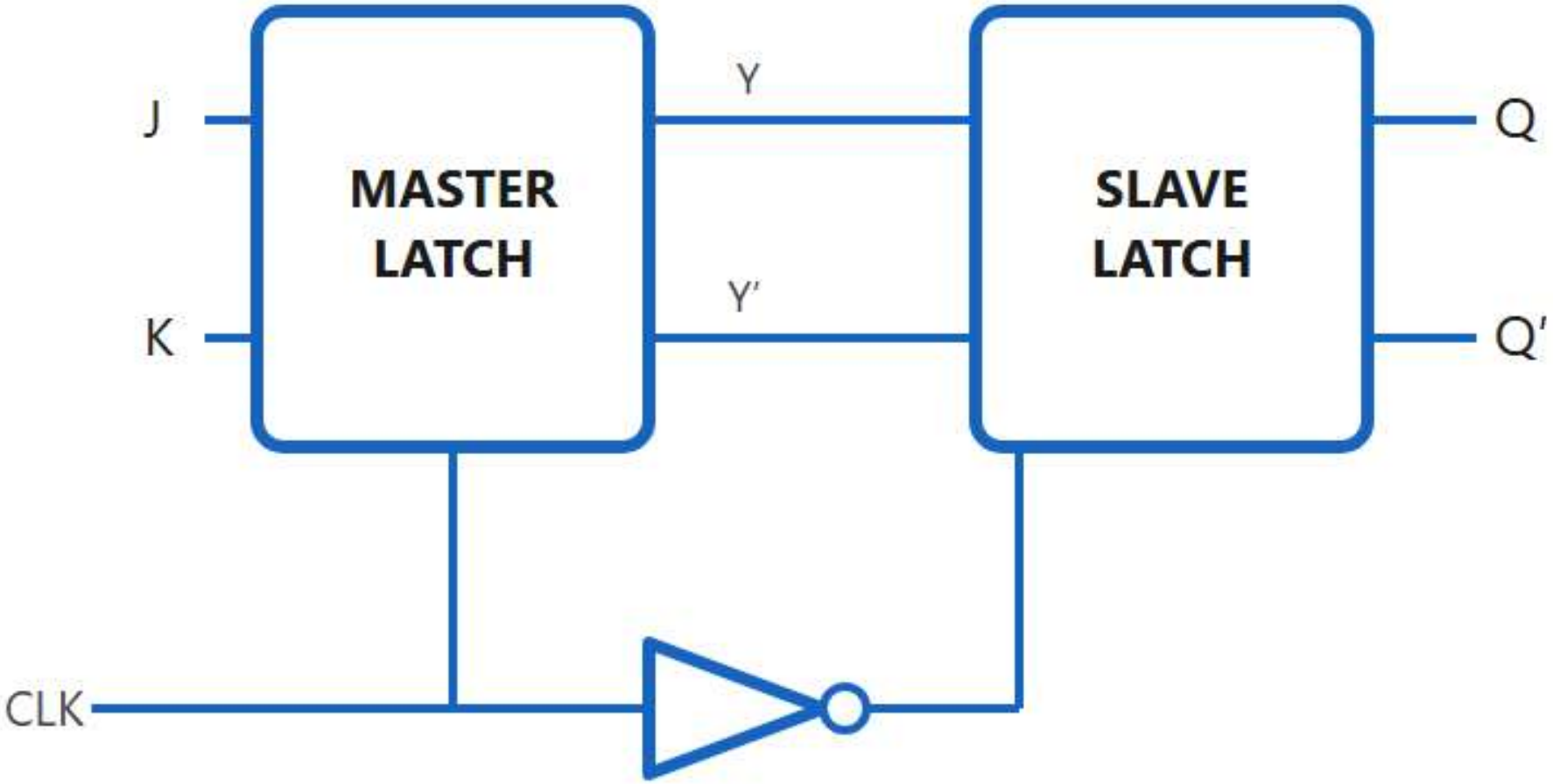
IN SIMPLE TERMS

A flip-flop is a latch that changes only at the clock edge, so every flip-flop in a system updates at the same instant.

Latch versus flip-flop; race-around; master–slave

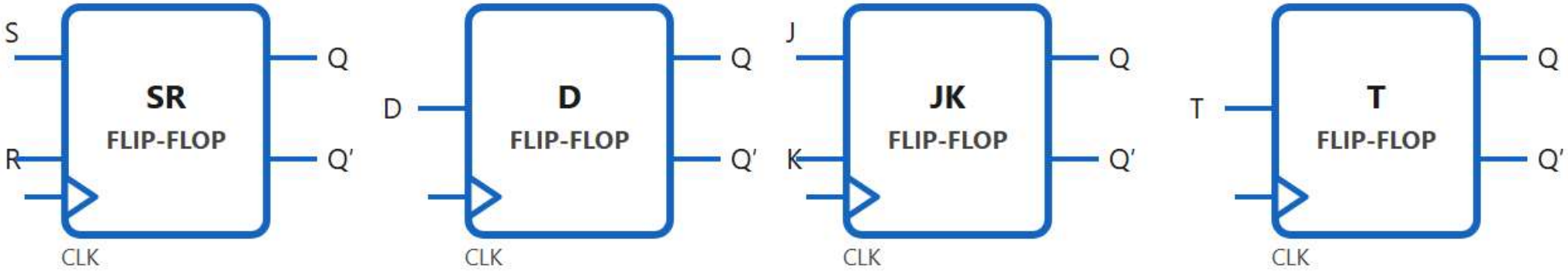
- A gated D latch is **level-sensitive**: Q tracks D the whole time EN is high
- A **flip-flop** is **edge-triggered**: it samples its inputs and updates Q only at the clock edge (↑ or ↓)
- **Race-around:** a level-triggered JK latch with J = K = 1 toggles again and again while the clock stays high; the final state is unpredictable
- **Master–slave fix:** two latches on opposite clock phases are never transparent together
- **CLK = 1:** master absorbs J, K into Y; slave is latched and isolates Q
- **CLK = 0:** master locks Y; slave becomes transparent and copies Q = Y: Q changes only on the falling edge

Master–slave JK flip-flop



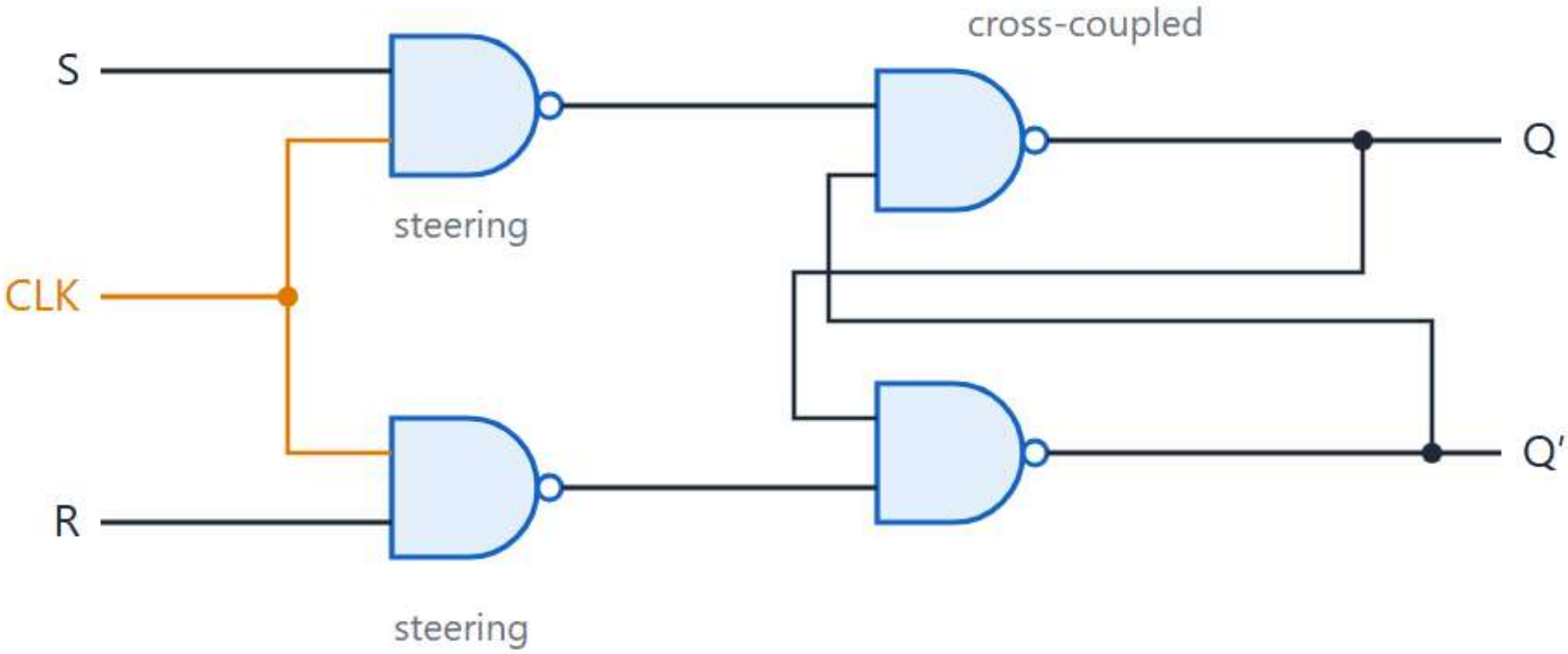
Master–slave JK flip-flop

Flip-flop block symbols



SR (Set/Reset), D (single data line), JK (no forbidden state), and T (Toggle) — the CLK triangle at bottom-left is present on all four.

Gate-level clocked SR flip-flop



While CLK=0, both steering gates output 1 regardless of S/R, so the cross-coupled pair simply holds its last state. While CLK=1, S and R (inverted by the steering NANDs) can set or reset the latch — which is exactly why this simple design still race-arounds if CLK stays high too long, and why the master-slave structure above is needed in practice.

Characteristic and excitation equations (1 of 2)

The characteristic equation gives Q_{n+1} from the inputs and Q_n; the excitation table gives the inputs needed for a wanted transition.

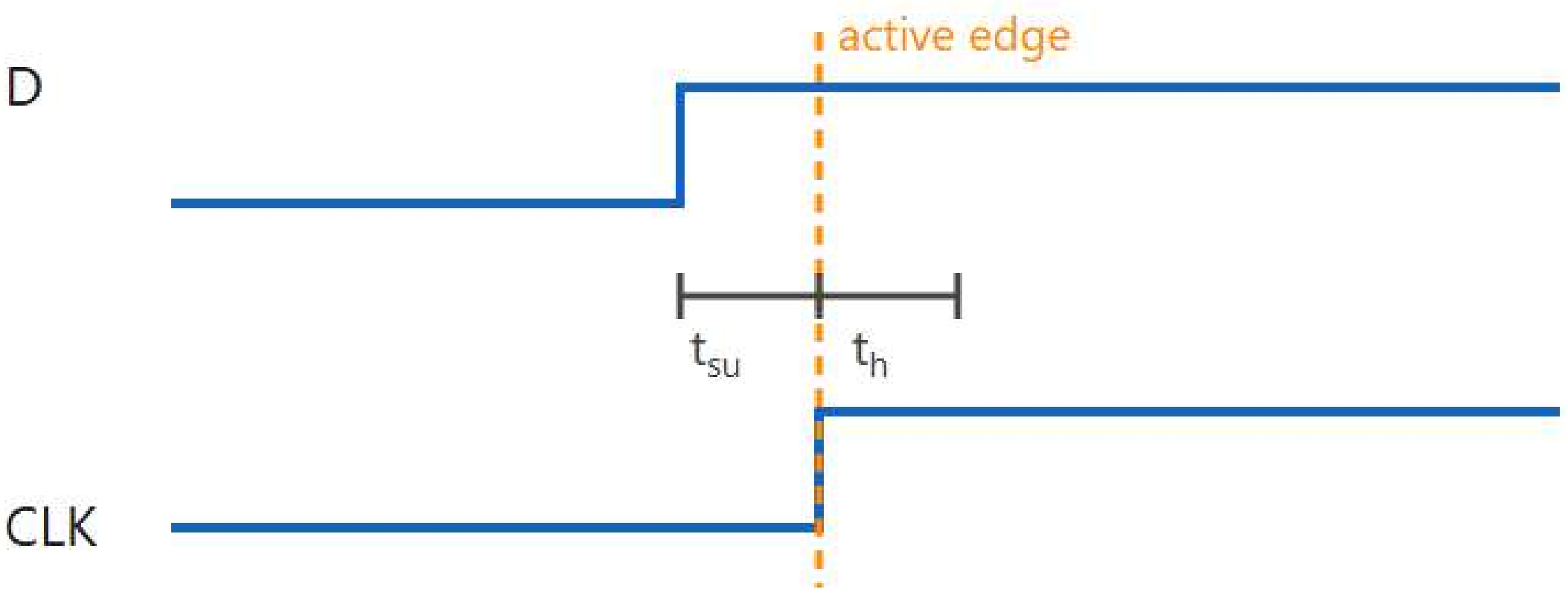
Type	Characteristic equation	$Q_n \rightarrow Q_{n+1}$	Required input(s)
D	$Q_{n+1} = D$	$0 \rightarrow 0$	D = 0
		$0 \rightarrow 1$	D = 1
		$1 \rightarrow 0$	D = 0
		$1 \rightarrow 1$	D = 1
T	$Q_{n+1} = T \oplus Q_n$	$0 \rightarrow 0$	T = 0
		$0 \rightarrow 1$	T = 1
		$1 \rightarrow 0$	T = 1
		$1 \rightarrow 1$	T = 0

Characteristic and excitation equations (2 of 2)

X (don’t-care) entries in JK are what a later K-map step exploits to get a smaller circuit. SR has don’t-cares here too; D and T do not.

Type	Characteristic equation	$Q_n \rightarrow Q_{n+1}$	Required input(s)
JK	$Q_{n+1} = JQ_n' + K'Q_n$	0 → 0	J = 0, K = X
		0 → 1	J = 1, K = X
		1 → 0	J = X, K = 1
		1 → 1	J = X, K = 0
SR	$Q_{n+1} = S + R'Q_n$ (SR = 0)	0 → 0	S = 0, R = X
		0 → 1	S = 1, R = 0
		1 → 0	S = 0, R = 1
		1 → 1	S = X, R = 0

Setup time, hold time and clock skew



Setup time, hold time and clock skew

Setup time, hold time and clock skew

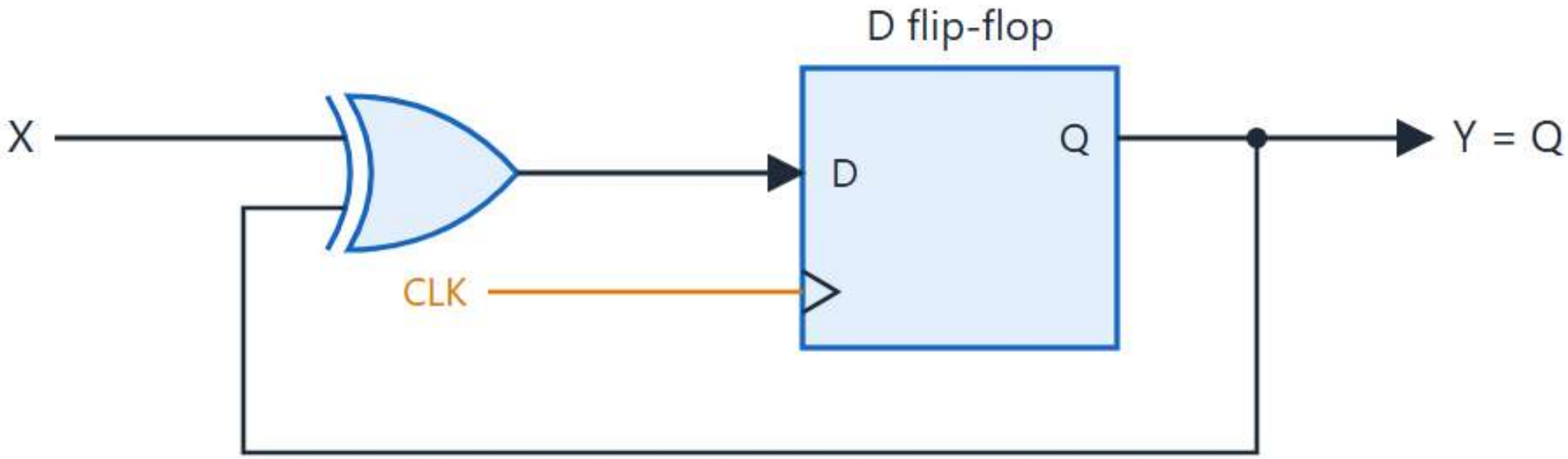
- **Setup time** (t_{su}): the minimum time D must be stable **before** the active clock edge
- **Hold time** (t_h): the minimum time D must stay stable **after** the active clock edge
- **Clock skew:** the difference in clock arrival time at different flip-flops, from unequal interconnect delay; it eats into setup and hold margin
- Violating setup or hold risks **metastability**: the output hovers between 0 and 1 before resolving unpredictably
- Real designs budget setup and hold margin explicitly

2.15 Analysis of Clocked Sequential Circuits

IN SIMPLE TERMS

To analyse a sequential circuit, write what each flip-flop input will be, then list every state and input to see where the circuit goes next.

Worked example circuit



D = X ⊕ Q, and the flip-flop's own Q feeds straight back as one of the XOR's inputs.

Next-state and output equations

Next-state equation: $D = X \oplus Q$

Output equation: $Y = Q$

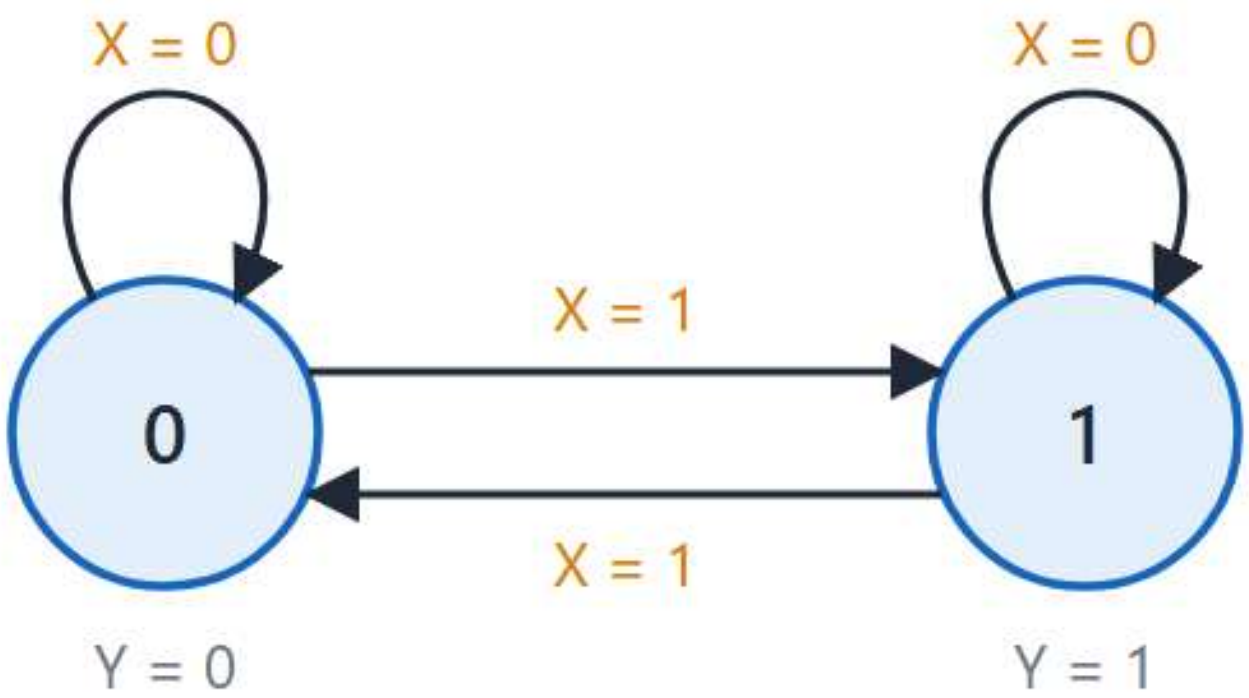
- D is what Q becomes after the next clock edge
- Step 1: write the Boolean expression feeding each flip-flop input (next state) and each external output
- One D flip-flop (present state Q), one input X, one output Y

Step 2: state table

Every present-state/input combination appears exactly once. Y depends only on the present state: a **Moore** machine.

Present State (Q)	Input (X)	Next State (D)	Output (Y)
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	1

Step 3: state diagram



State diagram — X=0 keeps the circuit in its current state, X=1 always flips it. Each circle's output Y is shown attached to the state, since Y depends only on present state here.

2.16 Registers

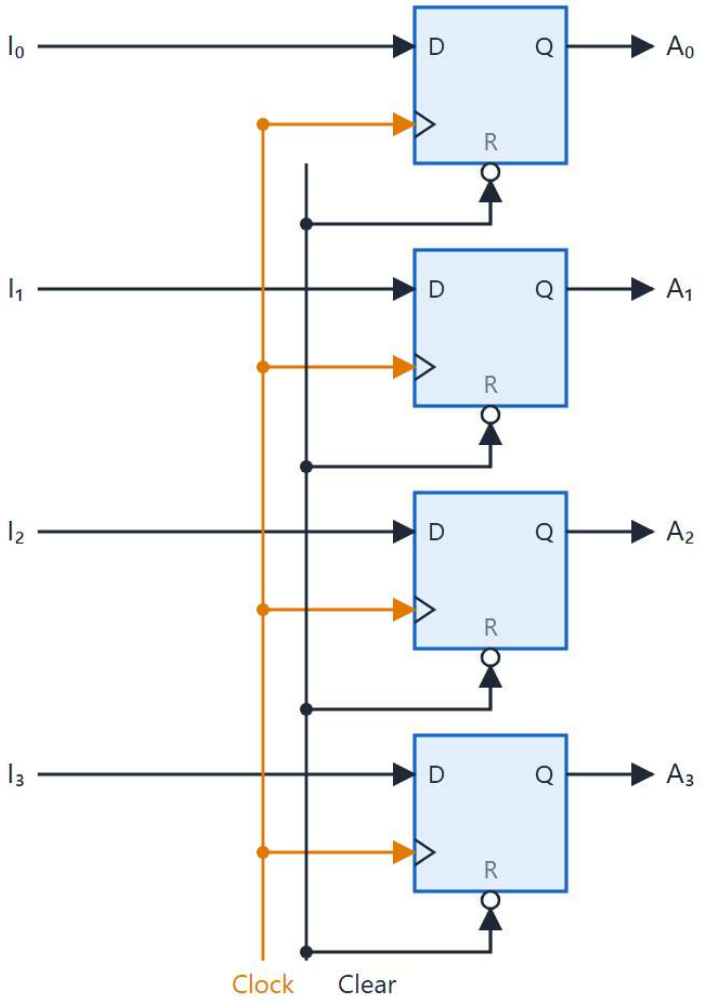
IN SIMPLE TERMS

A register is a row of flip-flops that store a binary word together, all updated by the same clock.

Registers and parallel load

- An **n-bit register** is n D flip-flops, one per bit, all clocked together
- Each D input has its own data line: bits load independently and in parallel
- On every active clock edge it captures its data inputs and holds them until the next edge
- A plain register captures new data on **every** clock edge
- Add a **LOAD** line and a 2-to-1 MUX at each D input: **LOAD = 0** selects Q (hold), **LOAD = 1** selects new data I (load)
- All bits share the same LOAD and clock: the register holds everything or loads everything

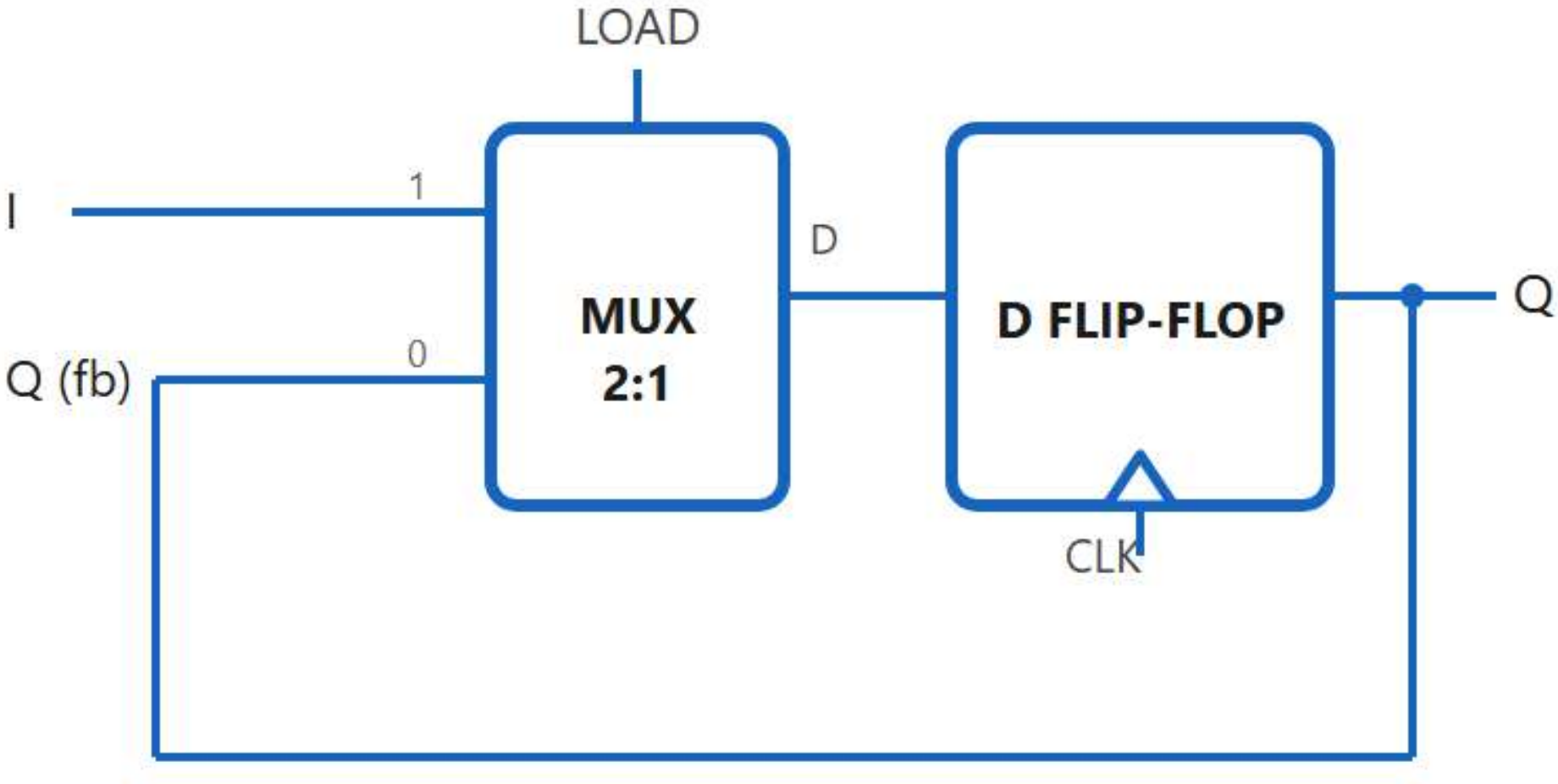
Four-bit register



- Four D flip-flops share one clock
- Each flip-flop has its own dedicated data line
- An active-low Clear input resets the register
- On every active clock edge the register captures its inputs and holds them until the next edge

Four-bit register: four D flip-flops sharing Clock, with an active-low Clear. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 6.1.

One bit of a register with parallel load



One bit of a register with parallel load. LOAD=0 selects input 0 (Q fed back, hold); LOAD=1 selects input 1 (new data I, load). Every bit repeats this exact circuit.

Worked example: hold, load, hold (Q = 0, I = 1 throughout)


```
Clock 1: LOAD=0 → MUX selects Q(=0) → D=0 → Q stays 0    (hold)
Clock 2: LOAD=1 → MUX selects I(=1) → D=1 → Q becomes 1   (load)
Clock 3: LOAD=0 → MUX selects Q(=1) → D=1 → Q stays 1    (hold)
```

- I = 1 is present on every clock edge, but only Clock 2 captures it

2.17 Shift Registers

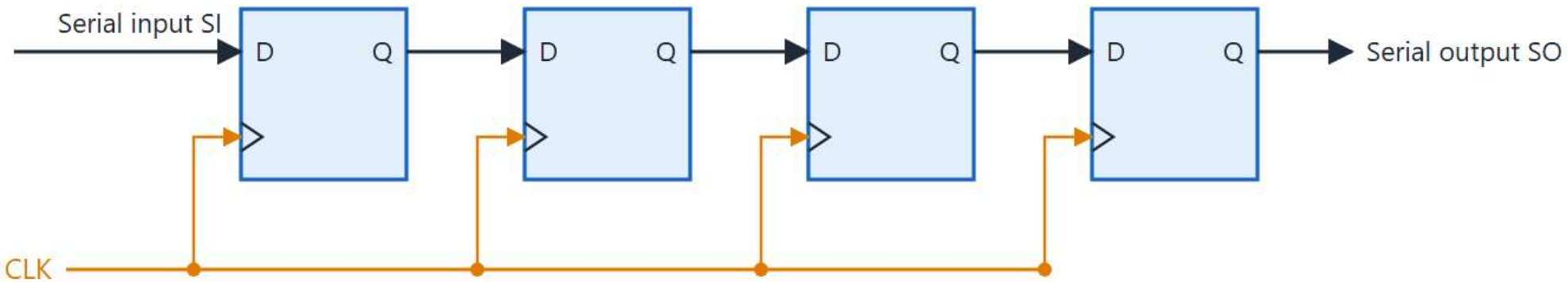
IN SIMPLE TERMS

A shift register passes its bits along one place per clock, like a bucket brigade for bits, so data can enter or leave one bit at a time.

Shift registers, SISO and SIPO

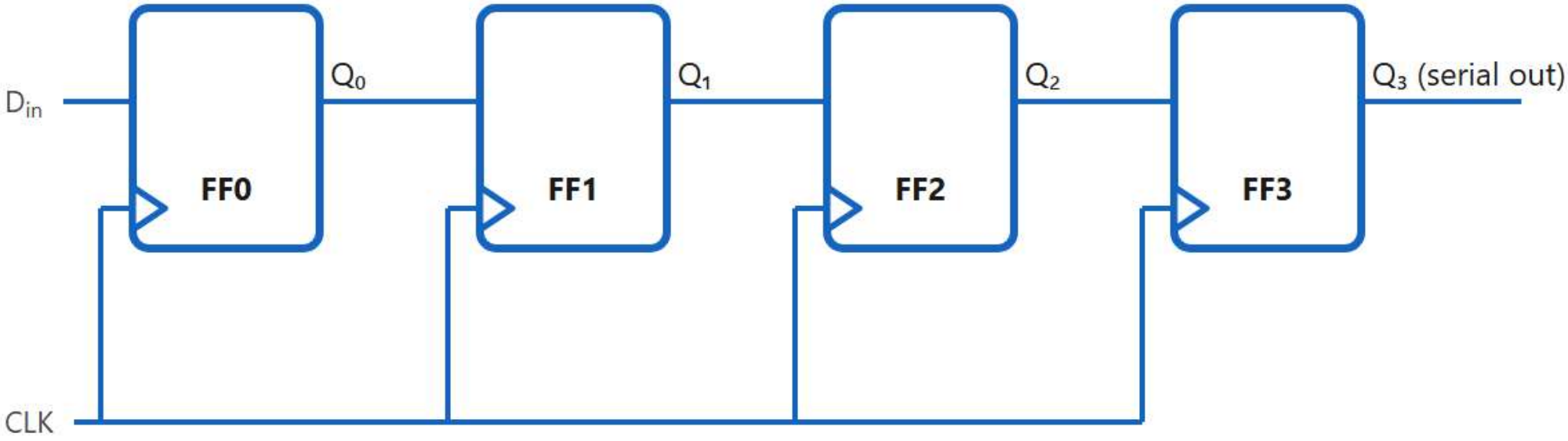
- A **shift register** is a cascade of flip-flops sharing one clock; each stage's D is the **previous** stage's Q
- On every clock pulse every stored bit moves one position down the chain
- Serial/parallel describes only how data goes in and out; the shifting structure never changes
- **SISO:** in one bit at a time, out one bit at a time from the last stage: delay lines and buffering (output = input delayed by N clocks)
- **SIPO:** same chain, every stage's Q tapped out: serial-to-parallel conversion, e.g. a UART receiver

Four-bit shift register



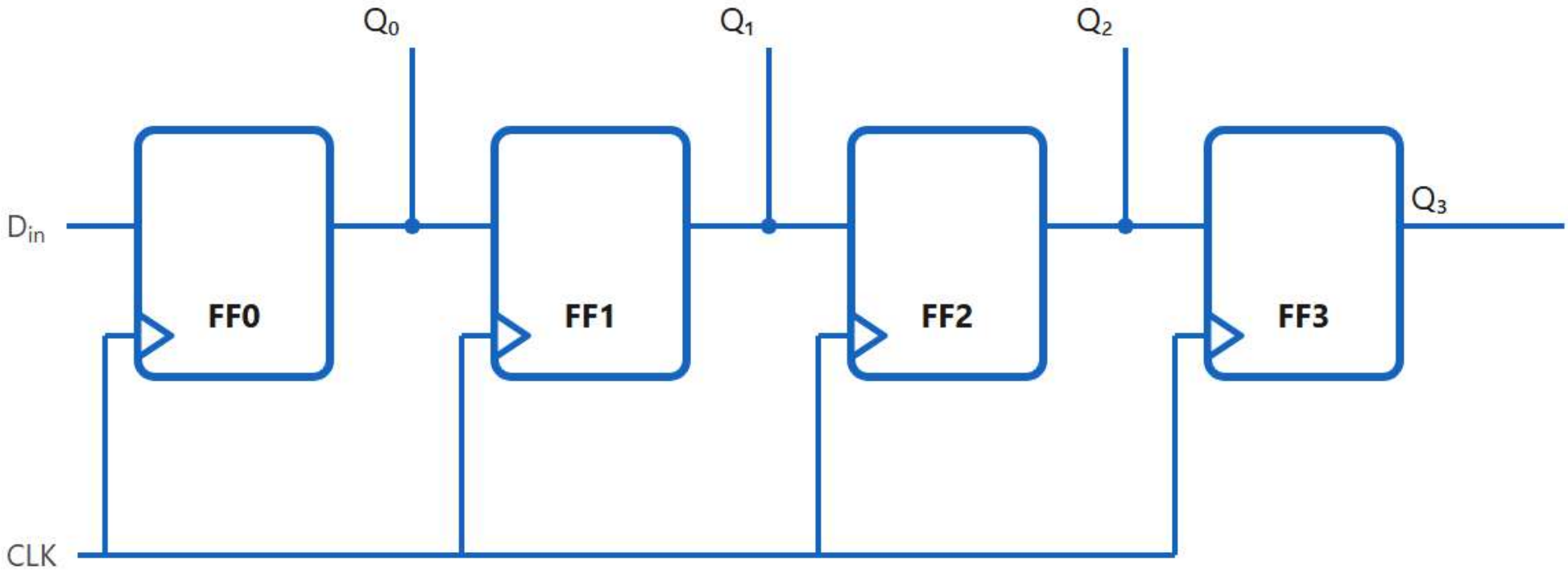
Four-bit shift register: each Q feeds the next D on every clock edge. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 6.3.

SISO: serial-in, serial-out



SISO: serial-in, serial-out

SIPO: serial-in, parallel-out

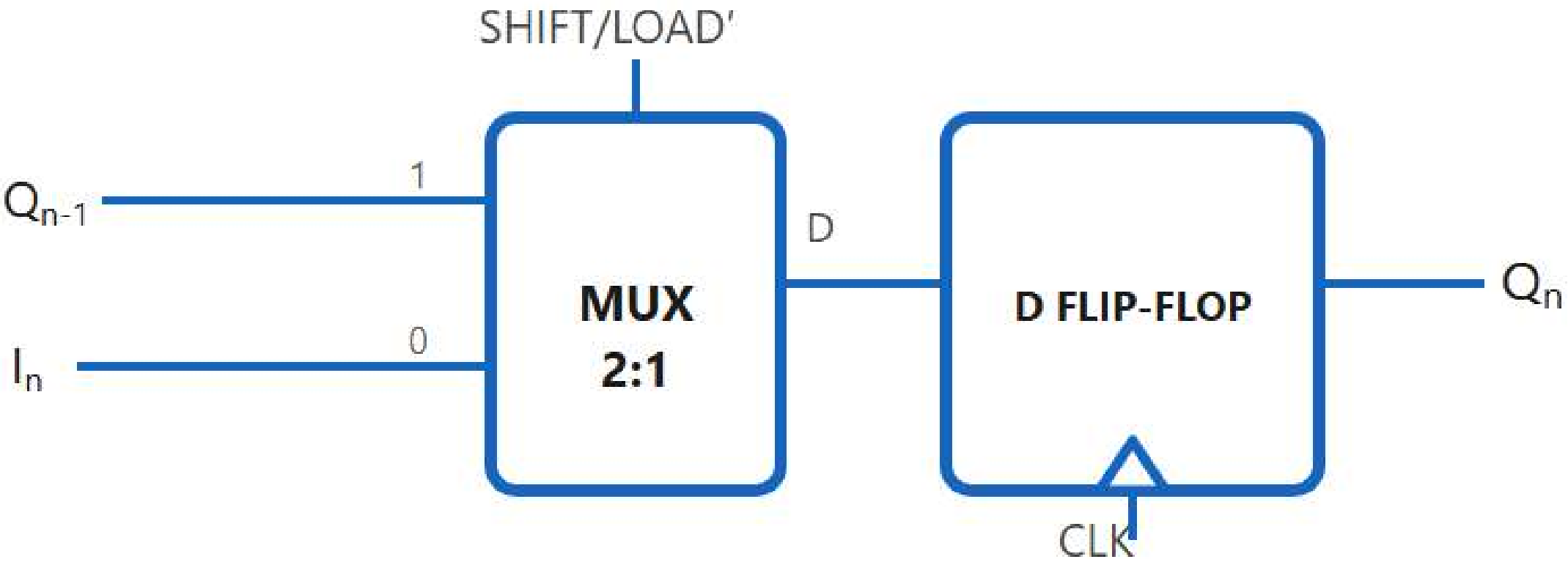


SIPO: serial-in, parallel-out

PISO and PIPO

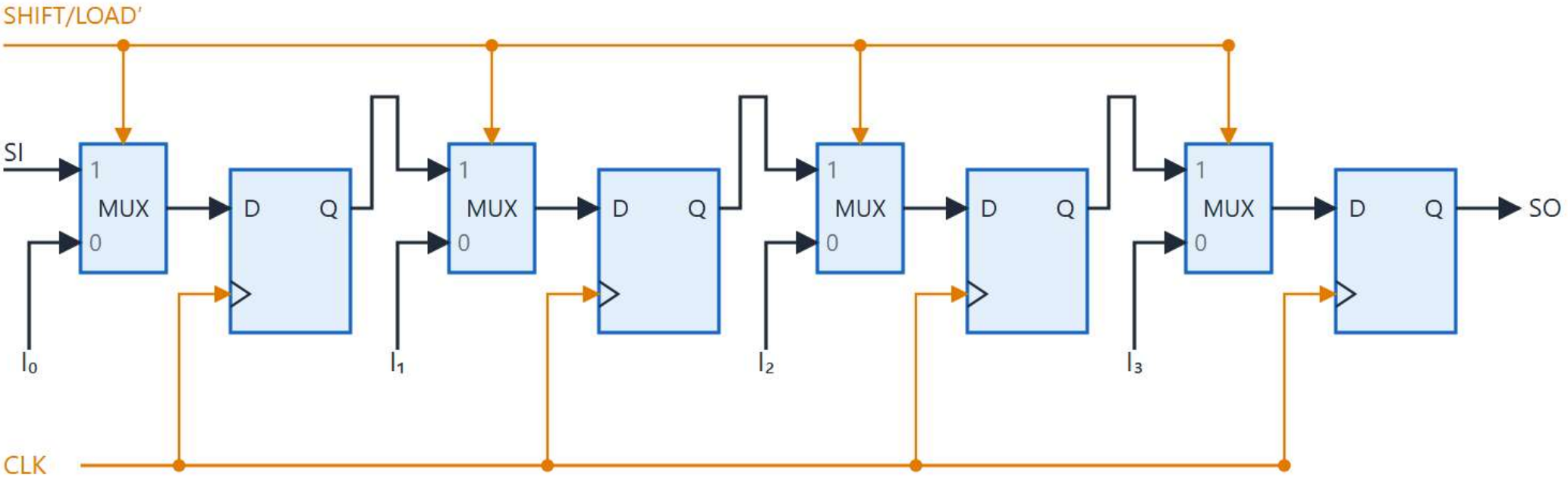
- **PISO:** a 2-to-1 MUX in front of each D chooses the previous stage's Q (shift) or an external parallel bit In (load)
- One shared SHIFT/LOAD' control line: **1** = shift, **0** = load
- Typical use: parallel-to-serial conversion, e.g. a UART transmitter loads a byte in one clock and shifts it out
- **PIPO:** all 4 bits load and read independently, with no connections between stages
- PIPO is functionally the plain register from the previous topic: the "no shifting" end of the family
- SISO and SIPO have no MUX and no control line: they only shift

One PISO cell



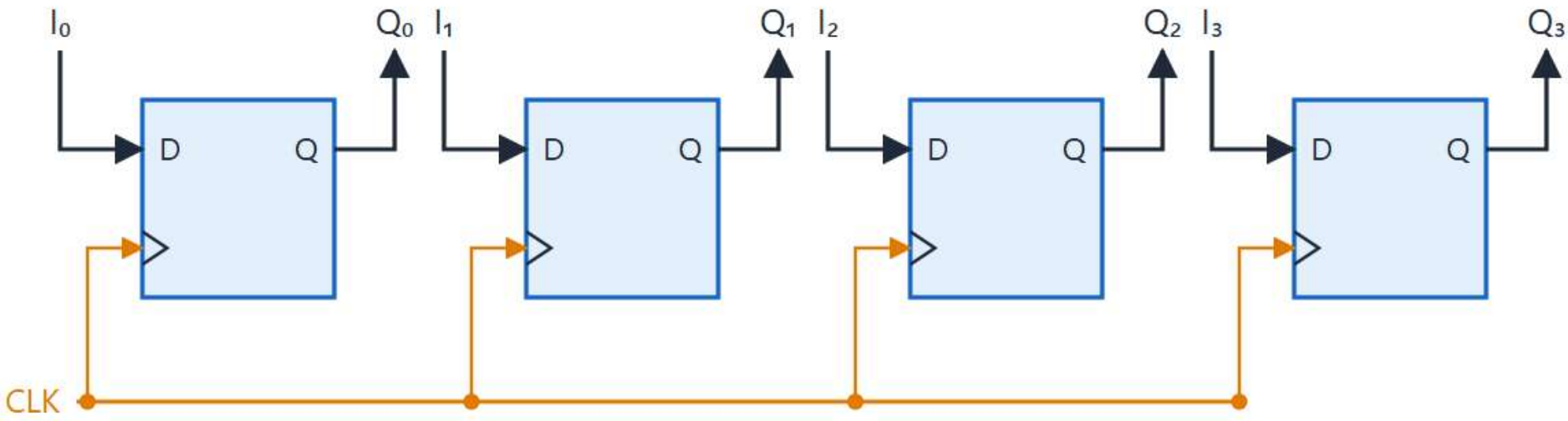
One PISO cell. SHIFT/LOAD'=1 selects Qn-1 (shift in from the previous stage); =0 selects In (load this stage's parallel bit directly). The final stage's Q is the serial output.

4-bit PISO register



SI (serial-in) ties the very first stage's shift input low when there's no earlier stage to shift from. I_0–I_3 are the four parallel-load pins, and every MUX shares the same SHIFT/LOAD' control line and CLK.

PIPO: parallel-in, parallel-out



Parallel-in, parallel-out: no connections between stages — all four bits load together on the clock edge.

Shift versus parallel-load operation

SHIFT/LOAD’	Each stage’s D input becomes	Result
1 (Shift)	Previous stage’s Q (serial-in for the first stage)	Data moves one position per clock
0 (Load)	External parallel bit I_n	All stages capture parallel data simultaneously

2.18 Ripple Counters

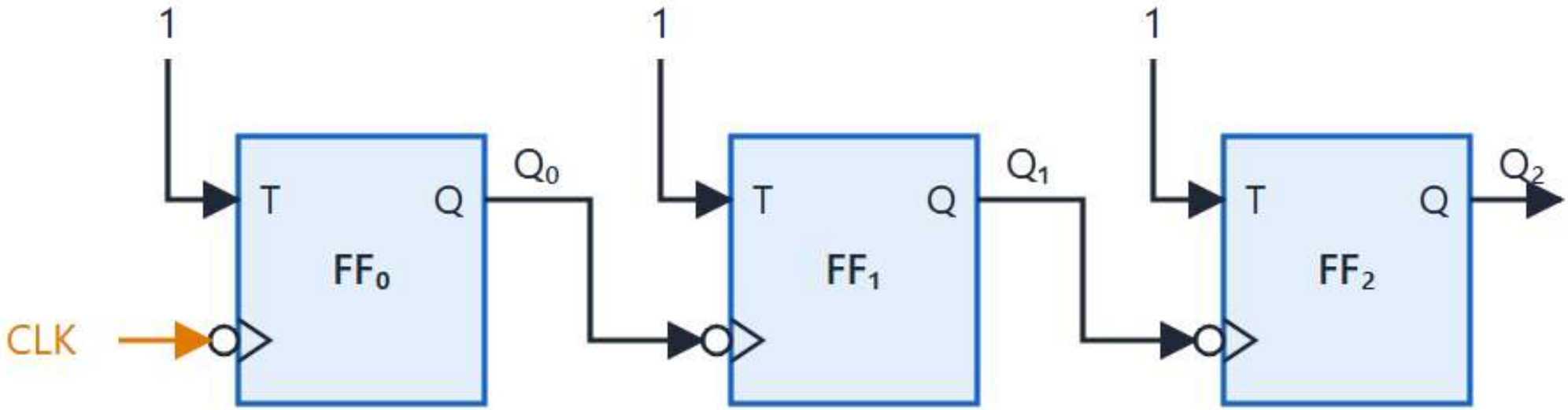
IN SIMPLE TERMS

In a ripple counter each flip-flop toggles the next one, so a count change ripples down the chain instead of happening all at once.

Asynchronous (ripple) up-counter

- In an **asynchronous (ripple)** counter only the first flip-flop gets the external clock; every later stage is clocked by the **previous stage's output**
- The new count ripples through the chain one stage at a time
- **3-bit up-counter:** negative-edge **T flip-flops**, each with T = 1
- FF0 (LSB) toggles every clock; FF1 toggles when Q_0 falls; FF2 when Q_1 falls
- Each stage runs at half the frequency of the one before: binary counting
- Sequence: 000 → 001 → 010 → 011 → 100 → 101 → 110 → 111 → 000 …

3-bit ripple up-counter



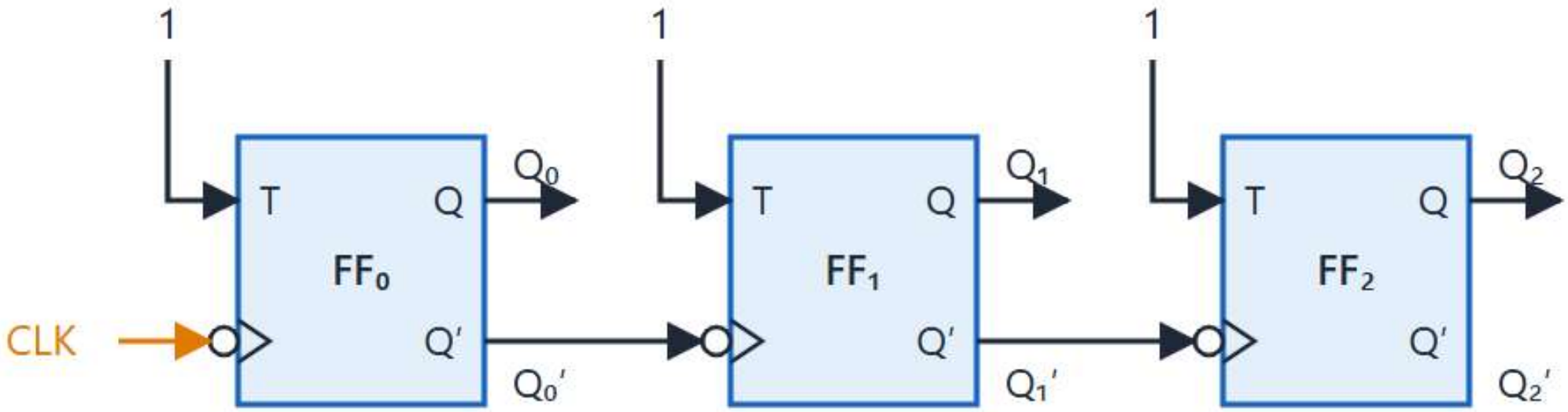
Three-bit ripple up-counter: negative-edge T flip-flops (bubble on the clock input), each clocked by the previous Q.

Up and down counter rules

Negative-edge + Q, or positive-edge + Q′, count up; the other two combinations count down.

Clock triggering	Next stage driven by	Counter mode
Negative edge (↓)	Q	**UP**
Negative edge (↓)	Q′	**DOWN**
Positive edge (↑)	Q	**DOWN**
Positive edge (↑)	Q′	**UP**

3-bit ripple down-counter



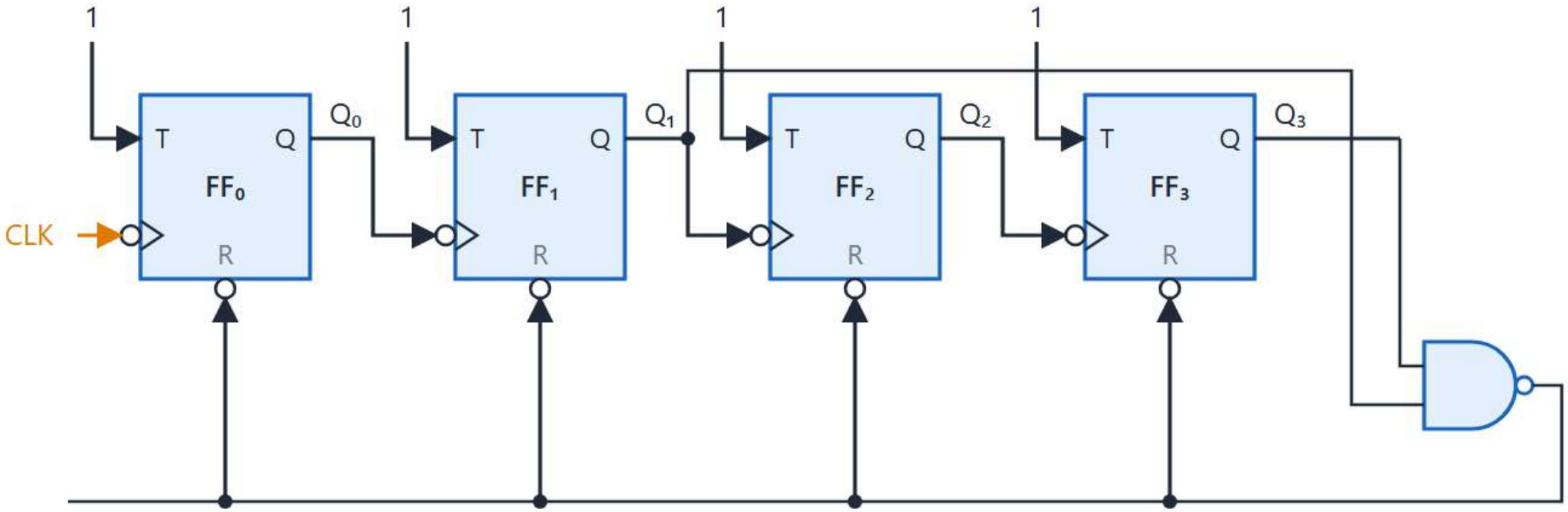
Three-bit ripple down-counter: the same chain, each stage clocked by the previous Q'.

Modulo-10 (BCD) ripple counter

Required FFs: $2^n \geq 10 \implies n = 4$ flip-flops (Q3Q2Q1Q0)
Reset state: Q3Q2Q1Q0 = 1010 (Q3 = 1 and Q1 = 1)
Clear logic: CLR′ = (Q3 · Q1)′

- A plain 4-bit ripple counter is mod-16; reset to 0000 the instant it would reach 1010
- The NAND output CLR’ drops to 0 and asynchronously clears every stage
- The brief 1010 can glitch the outputs; a synchronous clear after 1001 avoids it

Mod-10 ripple counter



Mod-10 ripple counter: a NAND gate detects 1010 ($Q_3 = Q_1 = 1$) and clears all four stages.

2.19 Synchronous Counters

IN SIMPLE TERMS

In a synchronous counter every flip-flop hears the same clock tick; gates decide which ones toggle, so all bits change together.

Synchronous counters: design procedure

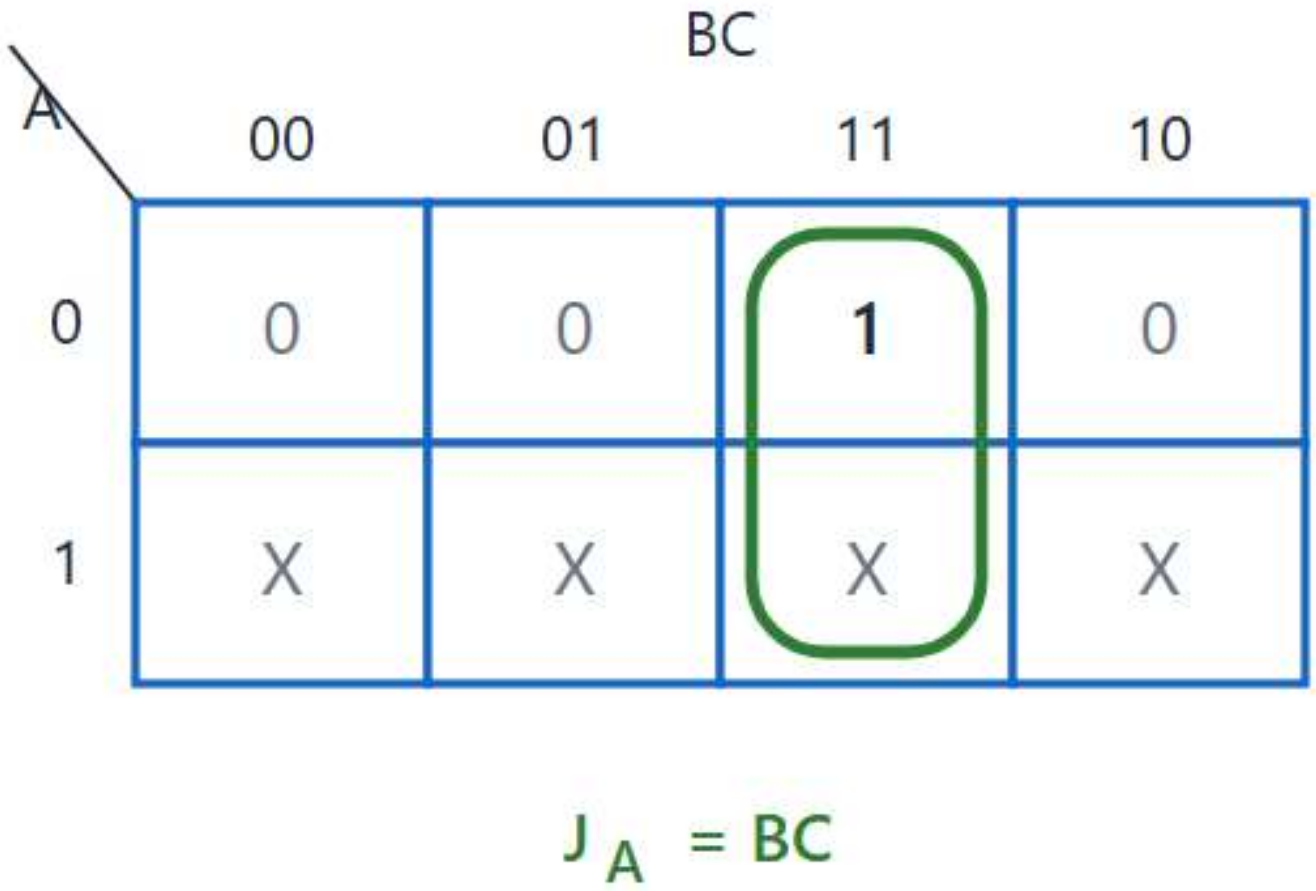
- Every flip-flop uses the **same clock**; no ripple delay accumulates
- Cost: extra combinational logic to compute each stage's inputs
- **1. State table:** each present state → required next state
- **2. Excitation table:** inputs each transition requires
- **3. K-map minimisation:** each flip-flop input is a function of the present state
- **4. Circuit:** wire the simplified logic and tie every clock to one common line

3-bit up-counter with JK flip-flops

Steps 1 and 2 combined: the required next state and the JK excitation values.

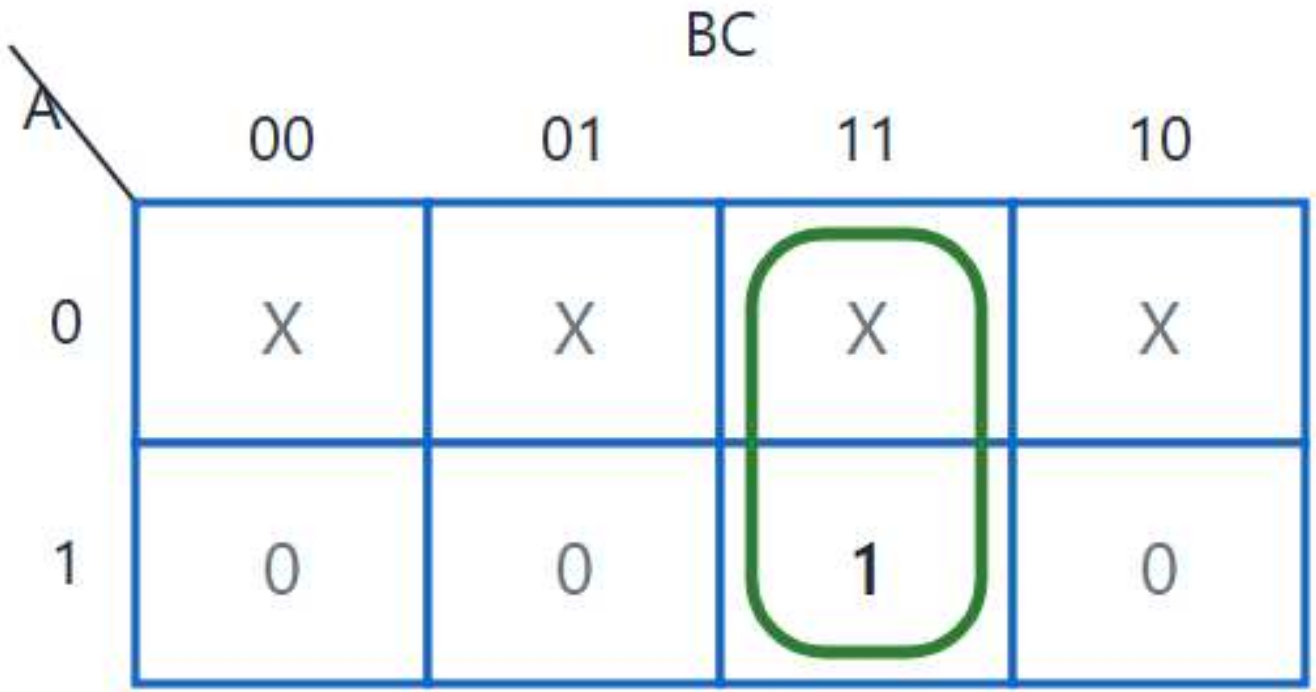
$A_nB_nC_n$	$A_{n+1}B_{n+1}C_{n+1}$	J_A,K_A	J_B,K_B	J_C,K_C
000	001	0, X	0, X	1, X
001	010	0, X	1, X	X, 1
010	011	0, X	X, 0	1, X
011	100	1, X	X, 1	X, 1
100	101	X, 0	0, X	1, X
101	110	X, 0	1, X	X, 1
110	111	X, 0	X, 0	1, X
111	000	X, 1	X, 1	X, 1

K-map for J of flip-flop A



K-map for J_A: the 1 at m3 groups with don't-care m7, giving J_A = BC.

K-map for K of flip-flop A



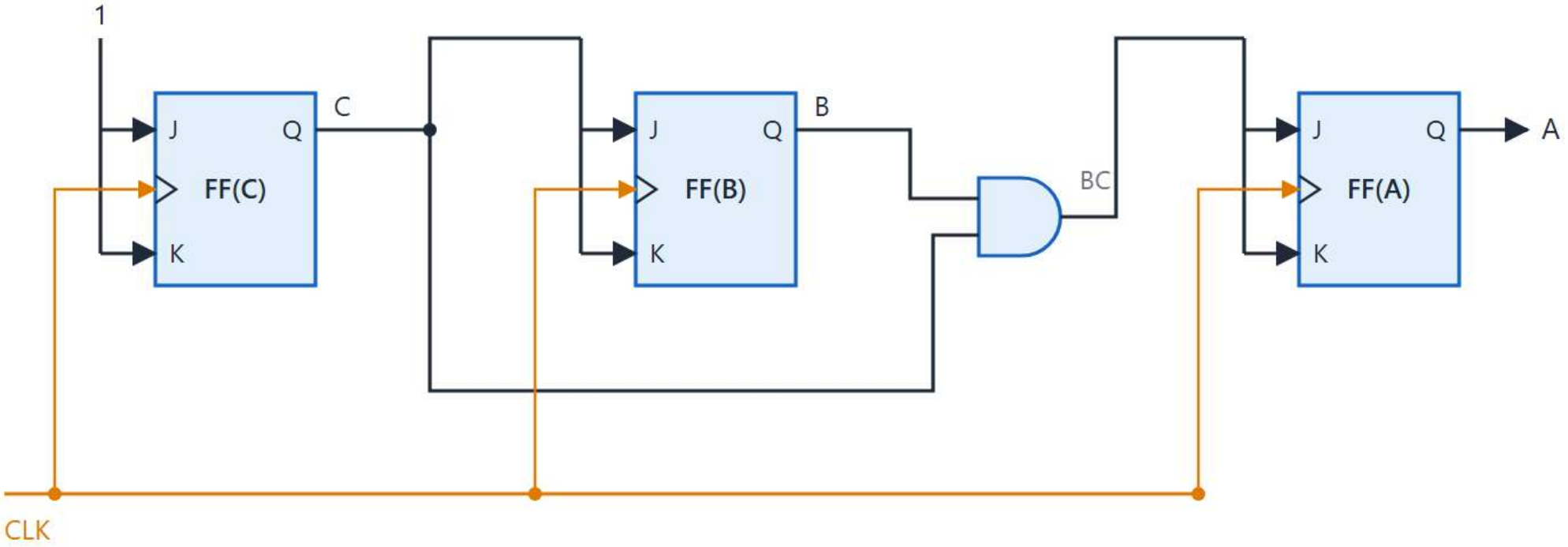
$K_A = BC$

K-map for K_A: the 1 at m7 groups with don't-care m3, giving $K_A = BC$.

Stages B and C; the circuit

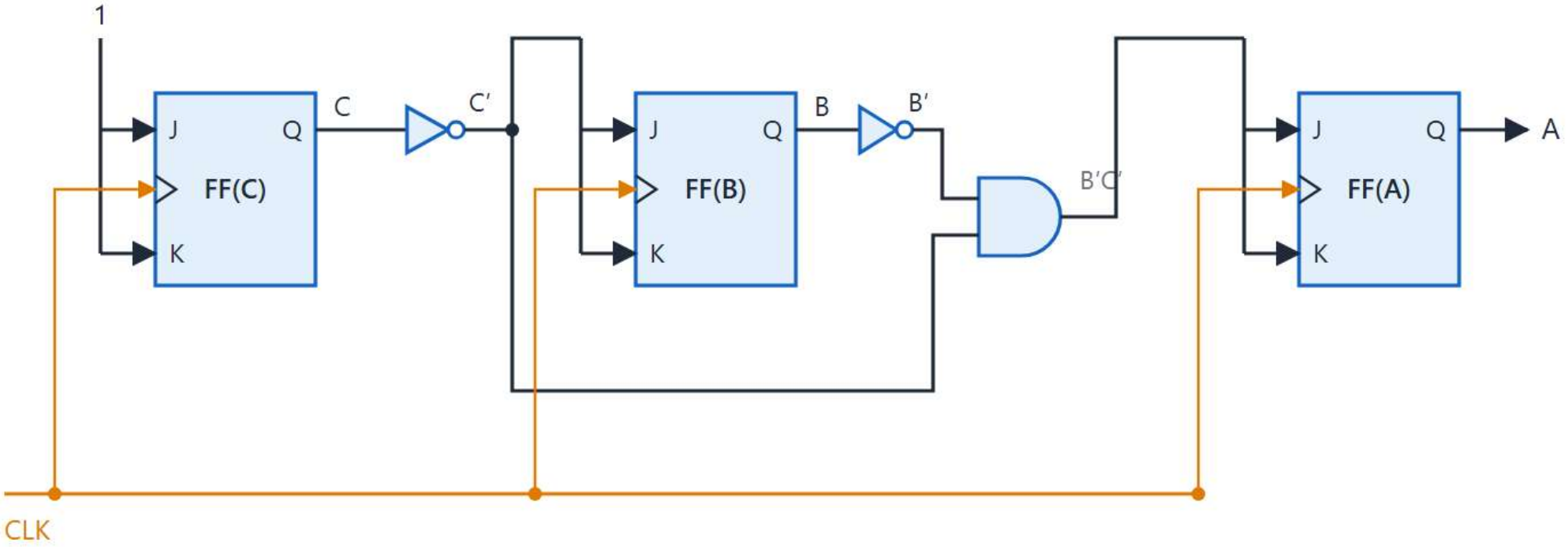
- Both maps group the same pair (BC = 11, both A rows): **$J_A = K_A = BC$**
- **$J_C = 1, K_C = 1$:** C toggles every clock, like the ripple counter's LSB
- **$J_B = C, K_B = C$:** B toggles precisely when C is 1
- In each stage J and K are tied together: "toggle when this condition is true"
- **Down-counter:** every toggle condition is the same with B and C inverted: $J_C = K_C = 1$; $J_B = K_B = C'$; $J_A = K_A = B'C'$
- Two inverters feed an AND gate in place of the up-counter's direct AND of B and C

3-bit synchronous up-counter



Three-bit synchronous up-counter: J and K of each flip-flop are tied together; all clocks share CLK.

3-bit synchronous down-counter



Three-bit synchronous down-counter: inverters supply C′ and B′; the AND gate forms B′C′.

2.20 Other Counters

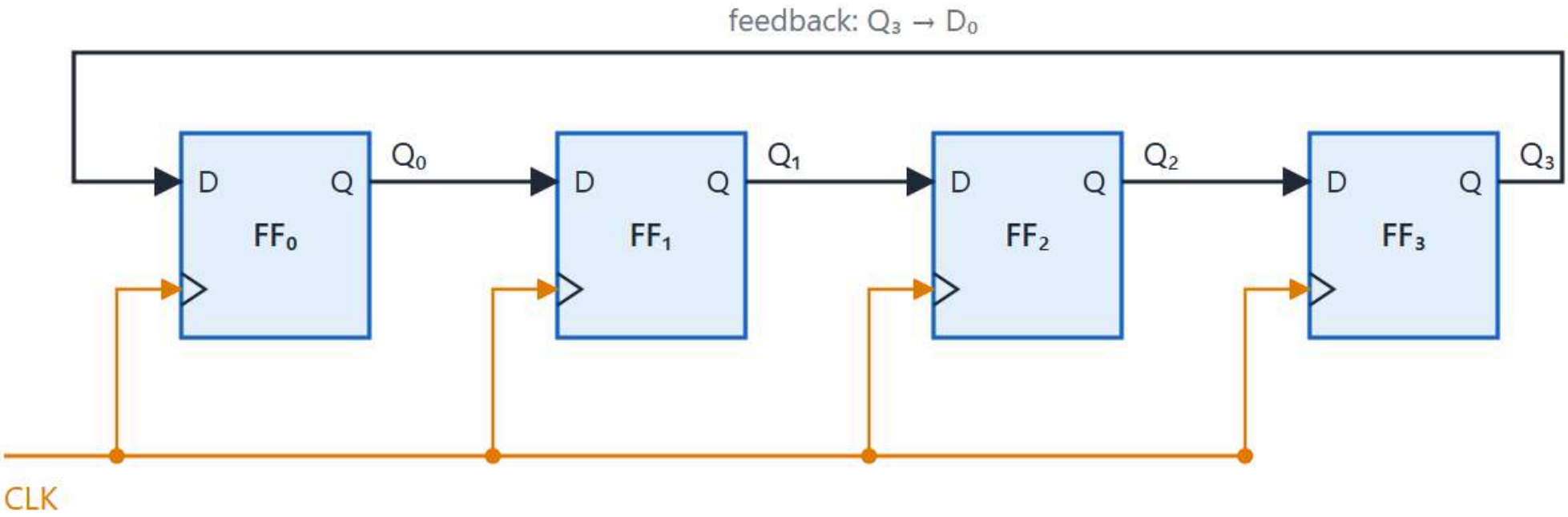
IN SIMPLE TERMS

A ring counter passes a single 1 around a loop of flip-flops; a Johnson counter feeds back the inverted bit, so it gets twice as many states from the same flip-flops.

Ring counter

- Feed the **last** stage's output back to the first stage's input of a shift register: no external serial data
- **N states** for N flip-flops: a single 1 (or single 0) circulates
- 4-bit sequence, pre-loaded to 1000: 1000 → 0100 → 0010 → 0001 → 1000 …
- **Must be pre-loaded:** reset gives 0000, a stuck state ($Q_3 = 0$ feeds $D_0 = 0$ forever)

Four-bit ring counter

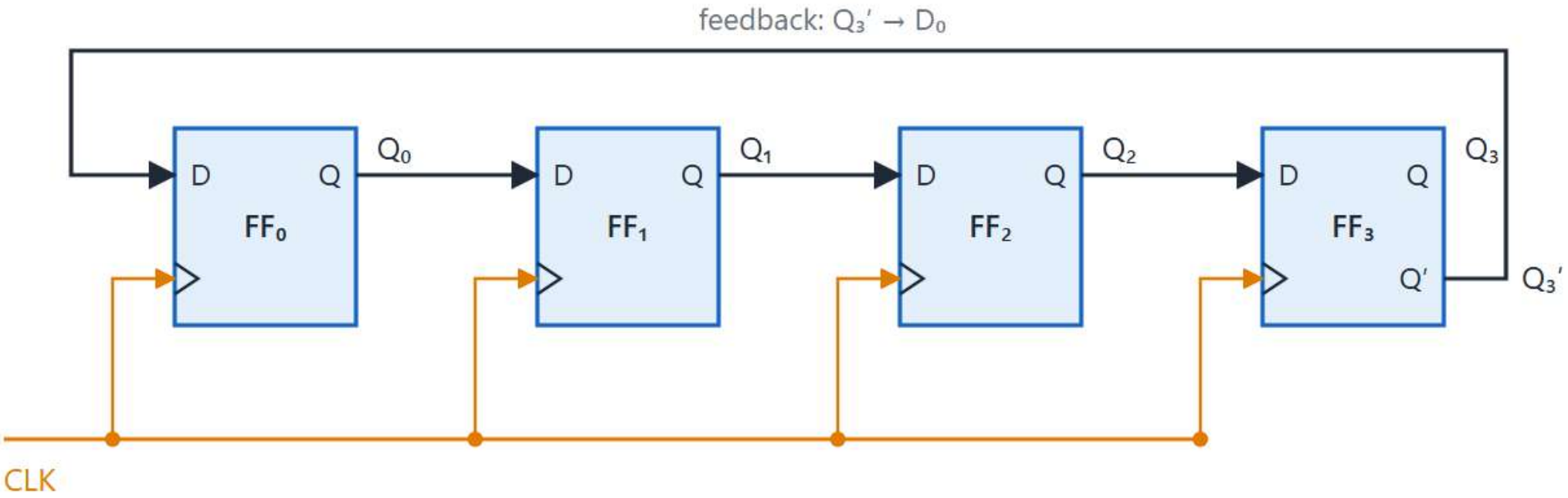


Four-bit ring counter: Q_3 feeds back to D_0.

Johnson counter (twisted-ring counter)

- Feed back the **inverted** last output: the circulating bit becomes a spreading and retreating band
- **2N states** for N flip-flops: twice the ring counter
- 4-bit sequence from reset (0000): 0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000 …
- **Self-starting** from 0000: $Q_3' = 1$ feeds $D_0 = 1$ on the next clock

Four-bit Johnson counter

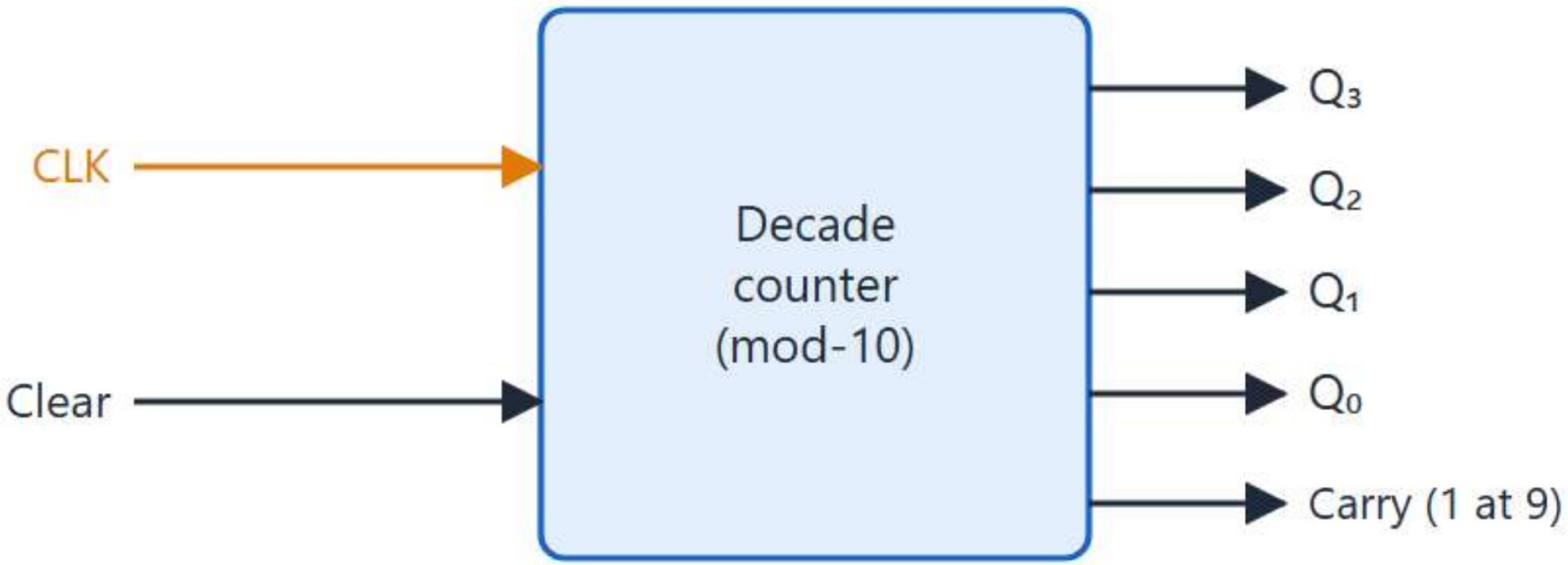


Four-bit Johnson counter: Q_3' feeds back to D_0.

Ring counter versus Johnson counter

Feature	Ring counter	Johnson counter
Number of states	N	2N
Feedback connection	$D_0 = Q_{N-1}$	$D_0 = Q'_{N-1}$
Initialization	Requires pre-loading (e.g., 1000)	Self-starting from reset (0000)
Decoding a state	Single bit is already the decoded output	Needs a 2-input gate per state to decode

Decade counter



Decade counter: counts 0000 to 1001, with a carry output at 9 for the next decade.

Unit 2 summary (1 of 2)

- **2.1 Combinational circuits:** output depends only on current inputs; no feedback
- **2.2 Analysis:** label gate outputs, write expressions, substitute, tabulate
- **2.3 Design:** specification, truth table, simplify, draw and verify
- **2.4 Binary adder:** half adder, full adder, ripple carry chain of full adders
- **2.5 Binary subtractor:** A − B = A + B′ + 1; XOR gates and M make an adder–subtractor
- **2.6 BCD adder:** binary add, then add 0110 when Output carry = $K + Z_8Z_4 + Z_8Z_2$
- **2.7 Binary multiplier:** AND gates for partial products, adders for the columns
- **2.8 Magnitude comparator:** compare from the MSB; the first differing bit decides
- **2.9 Decoders:** n inputs select one of 2^n outputs; each output is a minterm
- **2.10 Encoders:** output the code of the active input; a priority encoder picks the highest

Unit 2 summary (2 of 2)

- **2.11 Multiplexers:** select one of 2^n inputs; with constants on the inputs a MUX implements a function
- **2.12 Sequential circuits:** output depends on inputs and state; next state = f(inputs, present state)
- **2.13 Latches:** cross-coupled gates hold a bit; the gated D latch is level-sensitive
- **2.14 Flip-flops:** edge-triggered; characteristic and excitation tables; setup and hold time
- **2.15 Analysis of sequential circuits:** next-state equations, state table, state diagram
- **2.16 Registers:** D flip-flops sharing a clock; LOAD with a MUX gives hold or load
- **2.17 Shift registers:** SISO, SIPO, PISO and PIPO; SHIFT/LOAD' picks shift or load
- **2.18 Ripple counters:** each stage clocks the next; mod-10 by NAND reset at 1010
- **2.19 Synchronous counters:** one shared clock; J and K from state table, excitation table and K-map
- **2.20 Other counters:** ring (N states), Johnson (2N states), decade counter
- **Key applications:** adders in every ALU; decoders for memory address selection; MUXes for data routing; registers and counters in CPUs